

Welcome to **Zenva's Create Interactive Pages with Javascript and the DOM API** course! In this course, we'll be learning how to use **Javascript** to manipulate the **DOM**, or **document object model**. For this first lesson, we will simply be talking about the **DOM** and learning what it is. You can follow along with the **Intro to DOM Text** file in the **Course Files**.

What is the DOM?

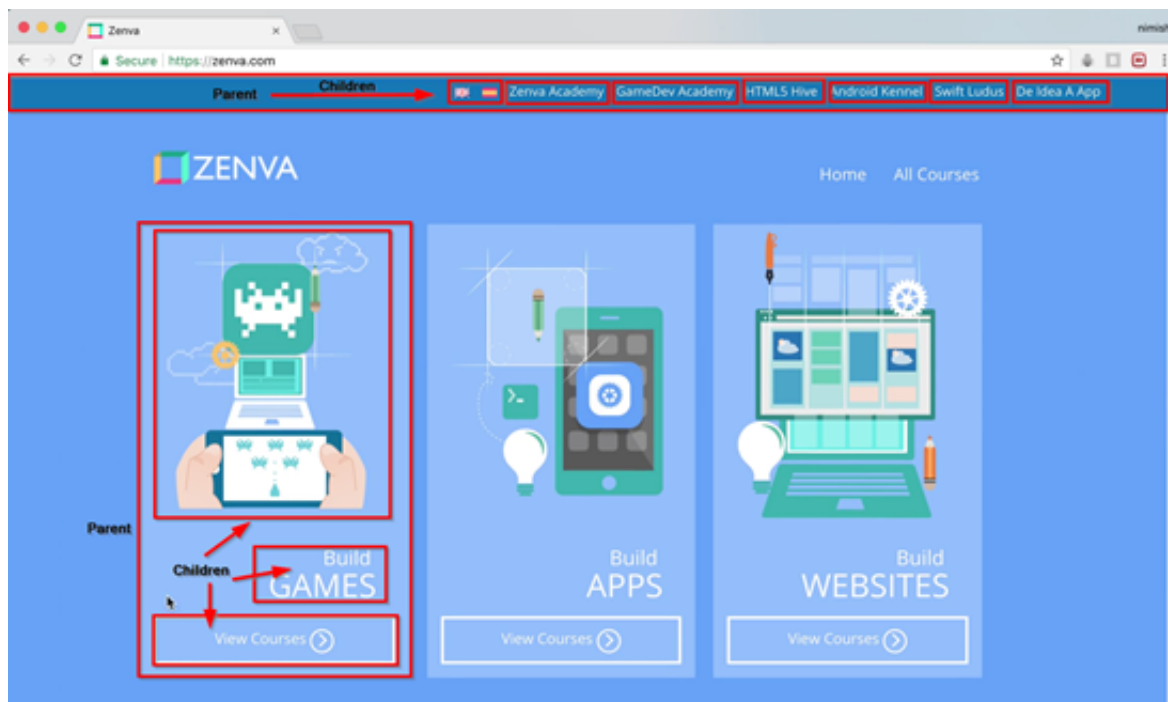
In **computer science**, one of the most fundamental ways to **structure data** is like a **tree**. The **DOM**, put simply, provides us a way to do this with **HTML**. The **document** itself is the **root/parent**, while all the other **HTML elements** are the **children/nodes** (branches of the tree, if you will). These **children** can have **HTML elements** that are **children** to them as well, though at the end everyone is directly or indirectly still a **child** of the **document**.

What this structure means for us is that we can easily manipulate and alter the web page at anytime using **Javascript**. This can include adding or removing elements, moving elements, and more. It is through this that our webpage stops being **static** and starts becoming **interactive**, since we can still access the page at run-time. Until recently, extra **libraries** weren't even around and one only had to use **script tags** to insert their interactivity. Overall, the **DOM** setup and **Javascript** provide an easy way to alter the **HTML**.

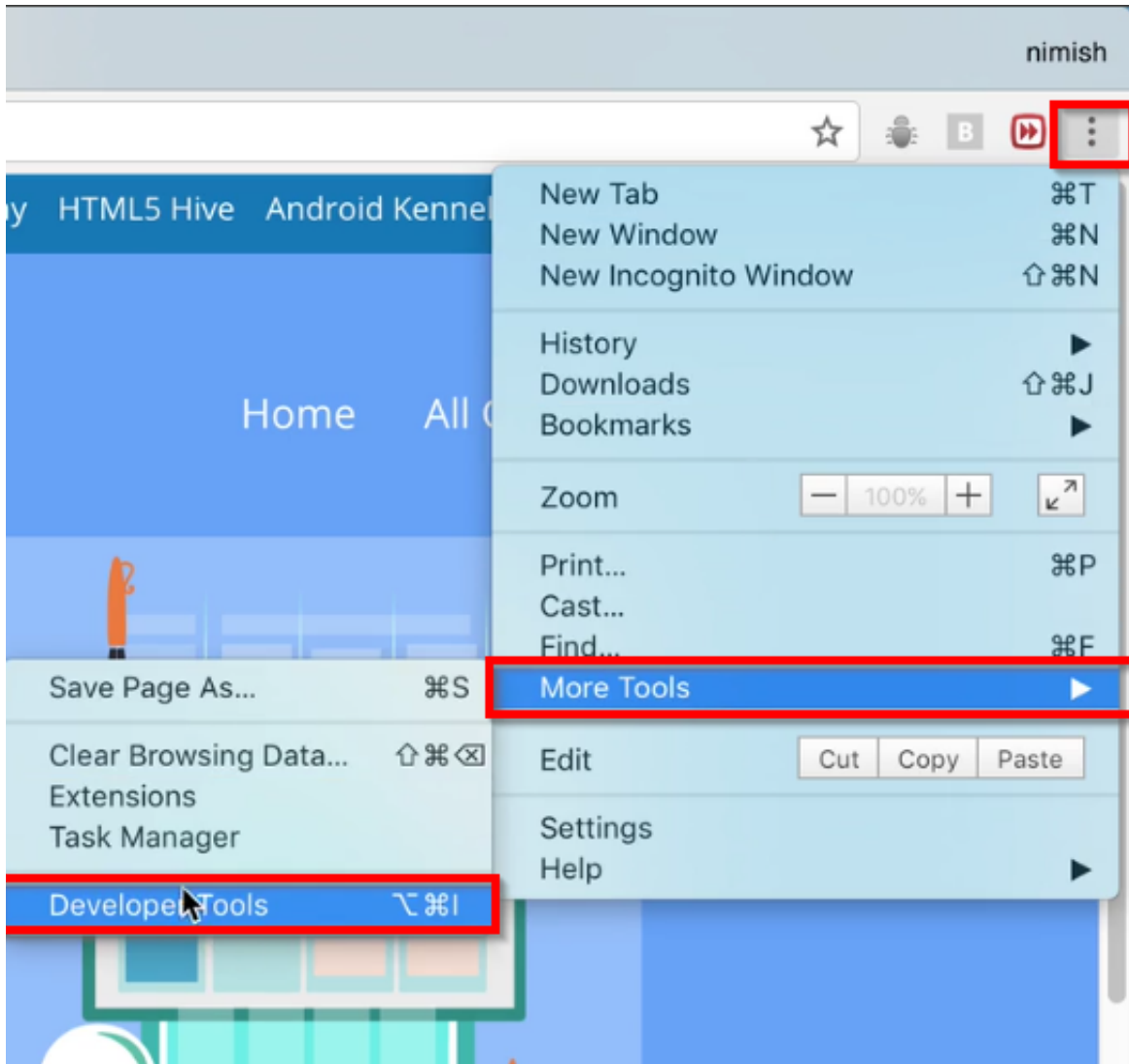
Examining Zenva.com

(**Note:** The lesson features an older version of the **Zenva** website (<https://academy.zenva.com/>) and does not reflect the current website's design).

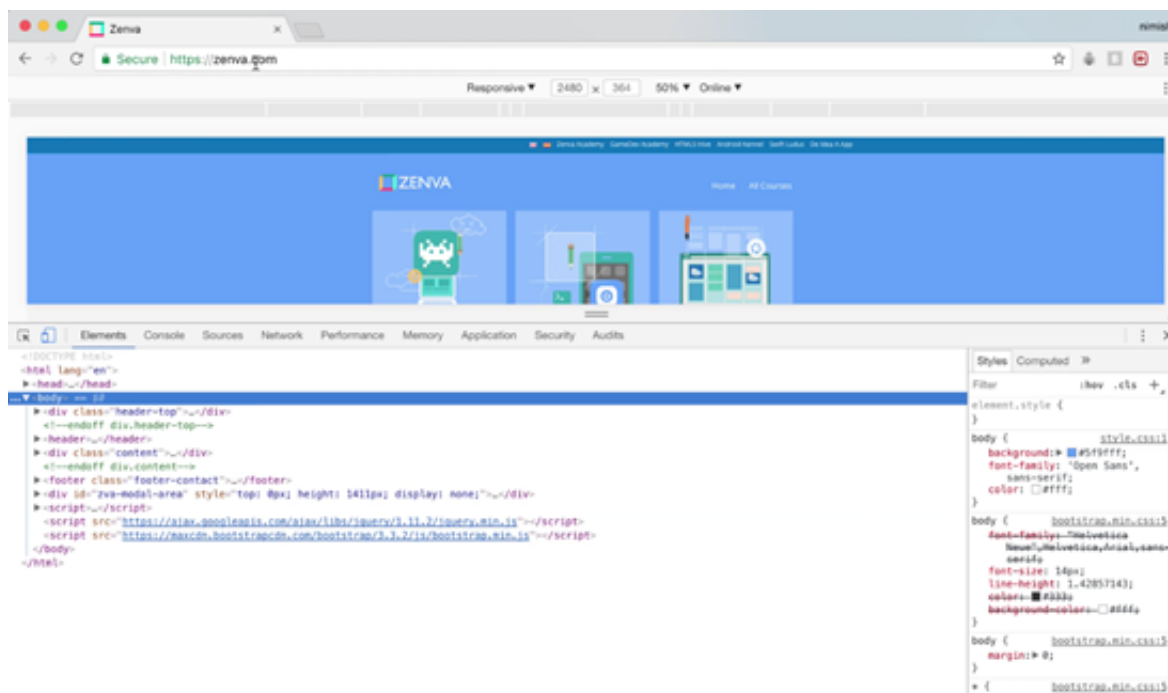
Let's take a look at **Zenva's website** to understand more about the structure. At the very top of the page you'll see a **navigation bar** which is a **root node** for all the **links**. However, this **navigation bar** is most likely a **child** of the **header**, who is in turn might be a **child** of a **container**. Further down the page you can see the **body** and **footer**, each with their own **child elements**.



We can also examine the code directly to see it in action. In **Google Chrome**, by clicking the **three dots** in the top right corner you can open up a menu. Further down this menu, you can select **More Tools** and then **Developer Tools**, which will bring up the **Inspector**.



Now we can take a look at the code! Keep in mind this technique works for any website that you may want to examine.



This **navigation bar** is the child of the **div row**, who is in turn the child of the **div container**, who in turn is the child of the **div header-top**. Above that is the **body**, who is then in turn is a **child** of the **HTML document** itself.

```
<body> <!--Child of the document-->
  <div class="header-top"> <!--Child of body-->
    <div class="container"> <!--Child of header top-->
      ::before
      <div class="row"> <!--Child of container-->
        ::before
        <nav class="menu-top col-xs-12 hidden-xs"> <!--Child of row-->
          <ul class="list-unstyled list-inline pull-right"> <!--Child of nav-->
            <li class="lang-en"></li> <!--Child of ul-->
            <li class="lang-es"></li>
            <li></li>
            <li></li>
            <li></li>
            <li></li>
            <li></li>
          </ul>
        </nav>
        ::after
      </div>
      ::after
    </div>
  </div>
</body>
```

Further down the code you can also see the use of the **script tags** which use **event listeners** to modify elements on the page even after the page has loaded.



```
><footer class="footer-contact">...</footer>
><div id="zva-modal-area" style="top: 0px; height: 1411px; display: none;">...</div>
▼<script>
  window.addEventListener("load",function() {
    if(document.body.clientWidth > 780) {
      document.getElementById('zva-modal-area').style.top = '0px';
      document.getElementById('zva-modal-area').style.height = document
      document.getElementById('zva-modal-area').style.display = 'block'
      document.getElementById('zva-modal').style.top = (document.docum

      document.getElementById('zva-modal-overlay').addEventListener('c
        document.getElementById('zva-modal-area').style.display = 'n
      });
      document.getElementById('zva-modal-close').addEventListener('cli
        document.getElementById('zva-modal-area').style.display = 'n
      });
    }
  });
</script>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/jquery.min.js"
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.2/js/bootstrap.min.js"
</body>
</html>
```

Practice

Before the next lesson, take some time to examine your favorite websites and see how the tree structure works on their pages.

In the next lesson, we're going to learn how to actually retrieve elements in our **DOM** so that they can be manipulated.

In this lesson, we're going to learn various methods to **access** the **elements** on our **DOM** using **Javascript**.

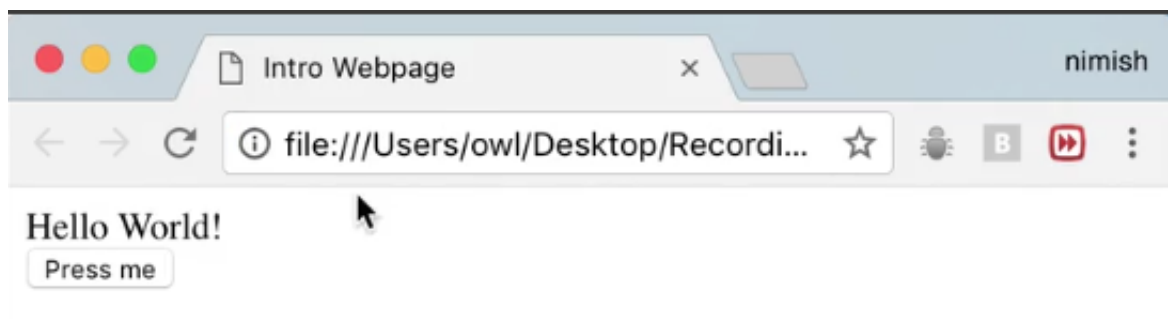
Creating Basic HTML

To begin, **create** a **New HTML file** in the **code editor** of your choice. For this course, we will be using **Atom** which you can download here: <https://atom.io/>. If using **Atom**, make sure to **set the file type to HTML**, as it defaults to **Plain Text**. We will be naming this file **Intro Webpage** and putting it into a project folder called **Intro to DOM**.

Now that we have our file, we can start creating some code. Our initial code will be fairly simple and consist of a **div** that has "Hello World!" and a **button**. The key for these elements, however, is to add **classes** and **IDs** to each of them. In this case, we'll give them the same **class** name but unique **IDs**.

```
<head></head>
<body>
  <div class='class1' id='div1'>
    Hello World!
  </div>
  <button class='class1' id='button1'>Press me</button>
</body>
```

Save the file once you complete this code. We're now going to **open it** up in our **browser** by locating the file where we saved it and **double clicking** it to open it.



First Script & Using Classes

Regardless of how you want to access elements on the **DOM**, all methods require the use of **scripts**, which we can add via **script tags** in the **body**. There are **five ways** in which we'll learn to **access elements**, the first being by **class name**.

Whenever we want to access an element, we need to first go in through a **root**, which typically will be the **document** in these cases. Since we're going for **class name**, we will use the function **getElementsByClassName** and specify we want **class1** from our code.

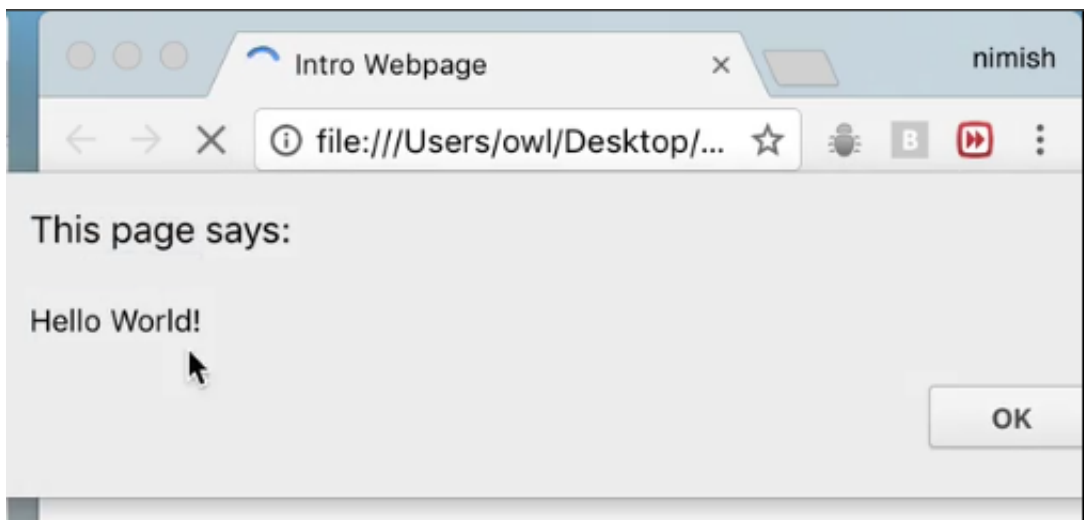
```
<script>
  var class1 = document.getElementsByClassName('class1');
</script>
```

Now, when our script runs, it will search the entire **document's** direct and indirect children to find this **class name**. Keep in mind that it can skip over parents. For example, in the code below, while the **parent div** won't be retrieved, its **child div** will due to it having the class.

```
<div>
  <div class="class1"></div>
</div>
```

Since we're using **getElements**, this means we've changed our **variable class1** into an **array**. As such, we can access specific elements by their **index** value. In the following code in our **script tags**, we create an **alert** for the **class1 variable** item at **index 0**. This would be our "Hello World!" **div** since it's the first saved element. Using **innerHTML**, we can access the **HTML** that's in that **div**.

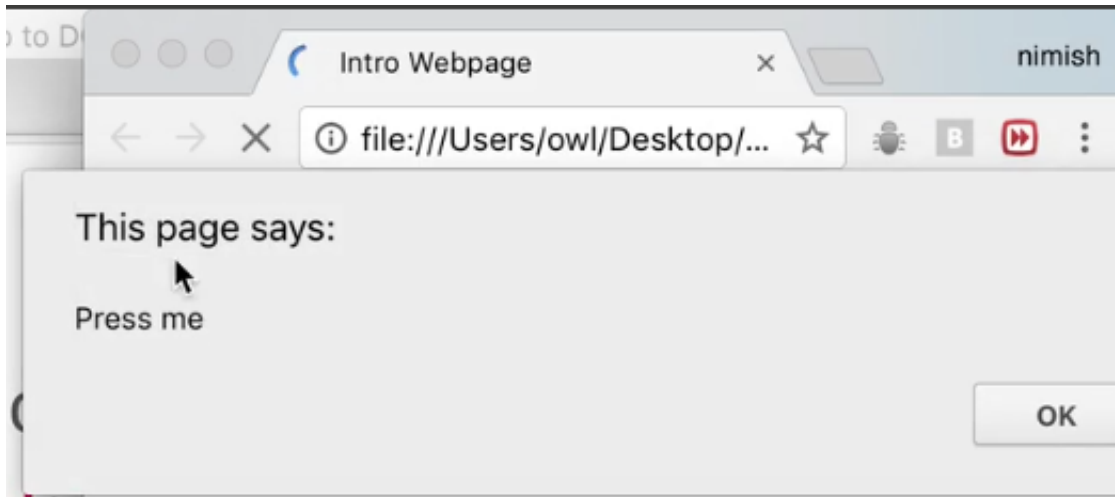
```
alert(class1[0].innerHTML);
```



Getting Elements by ID

Similar to classes, we can also access elements by their **IDs**. Since **IDs** are intended to be unique, we use **getElement** instead of **getElements** like we did for class. Again, though, all we need to do is pass in the **ID name** to get the element we want. Unlike our previous **variable**, this **variable** contains only a single value.

```
var button1 = document.getElementById('button1');
alert(button1.innerHTML);
```



Getting Elements by Tag Name

The next way we can access elements is by **tag name**. Within **HTML**, we use a variety of **tags** like **div**, **script**, **body**, **button**, and so forth. We can use those **tag names** to our advantage and get elements by them. In the example below, we will get all the **divs** on our page. Like **class**, we will be saving multiple elements, so the **divs variable** is an **array**. When using **tag names**, it's important to use *all caps* to access them properly.

```
var divs = document.getElementsByTagName('DIV');  
alert(divs[0].innerHTML);
```

Getting Elements by QuerySelector

Two other ways we can access elements on the **DOM** is by **queryselectors**. These methods are particularly powerful since they essentially combine all the previous methods into one. Not only can we search for elements by **class** or **ID** with these methods, but we can search for **attributes** like **color**. These methods, while similar, do have separate functionality. Plain **querySelector** only finds the **first** element that fits the parameter, while **querySelectorAll** finds all the elements that fit the parameter (and is thus an **array** for our saved variable).

Notice, however, that in previous examples we didn't use **.** or **#** to indicate our **classes** or **IDs**. This is because our functions already knew which it was looking for. When using **querySelectors**, it's important to use the correct symbol so it knows whether to look for a **class** or **ID**.

```
var first = document.querySelector('.class1');  
var all = document.querySelectorAll('.class1');
```

Nested Elements

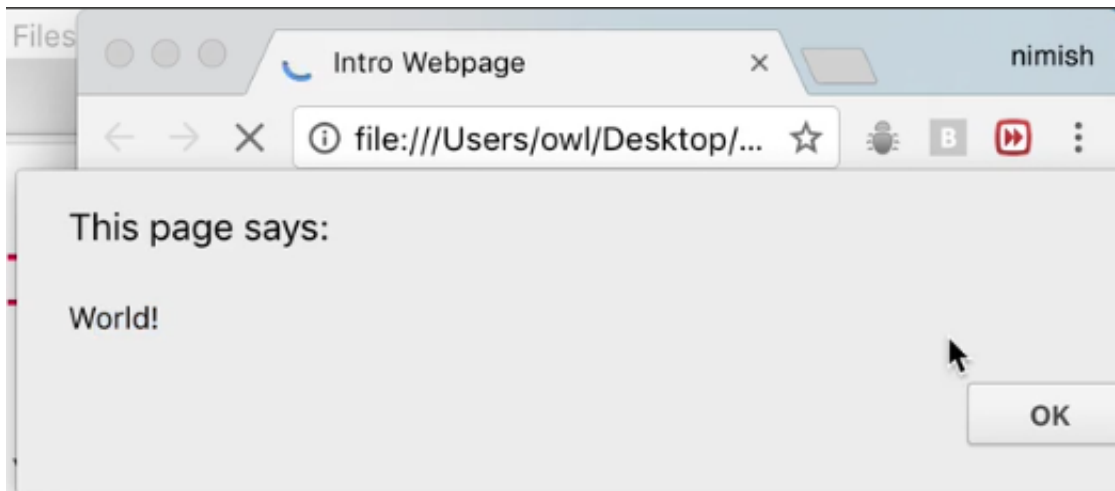
Our HTML didn't have many **nested** elements, so let's alter our HTML so we can see it in action. To do this, we're going to move "World!" into its own **div** after "Hello".

```
<div class='class1' id='div1'>  
  Hello  
  <div class='class1'>World!</div>  
</div>
```

</div>

Now let's create some code for our **script**. We will first create a pointer that saves our first **div** by the **ID div1**. Since this is saved, we can use this saved **variable** as a **root** to then access the next **div(s)** by **class name**.

```
var div1 = document.getElementById('div1');  
var innerDivs = div1.getElementsByClassName('class1');  
alert(innerDivs[0].innerHTML);
```



By running the code, you can see that we only get an **alert** for “World!” since we aren’t traversing the entire **document** but specifically the **div1 div**. Thus, we are able to completely ignore other mentions of the **class** on other branches.

Summary

Overall, we have five different ways to retrieve elements on the **DOM**:

- By class name
- By ID name
- By tag name
- By queryselector to get the first matching item to the query
- By querySelectorAll to get all matching items to the query

We can use these methods not only for the entire **document**, but for different **roots** in the document.

In the next lesson, we'll learn how to access and modify **attributes** using **Javascript**.

In this lesson, we're going to learn how to retrieve and modify the **attributes** of our **elements**. This includes **class** and **ID names**, **CSS styling**, or even the **inner HTML** itself.

Quick Styling and Why Javascript

Before we jump into altering anything, we're going to add a bit of **CSS styling** to our "Intro Webpage" page. In the **head tag**, we'll add a **style tag** and add a simple style that changes the background of **div1** to green.

```
<head>
  <style>
    #div1 {
      background-color: #00ff00;
    }
  </style>
</head>
```



Additionally, you might find it helpful to **comment out** all the previous lesson's code using **/*** and ***/**.

We can just about move onto the coding, but you might be asking why use **Javascript** when we can style everything in the **HTML**. Remember that with **Javascript**, we can change things at run-time and in response to events. This is not something you can do with **HTML** or **CSS**, since those create static elements. This method allows us to add more interactivity and interest for the user.

Retrieving Attributes

Let's start working with our example code in the **script tags**. The first thing we'll do is save our **div1 ID** since this is where a lot of stuff is happening.

```
div1 = document.getElementById('div1');
```

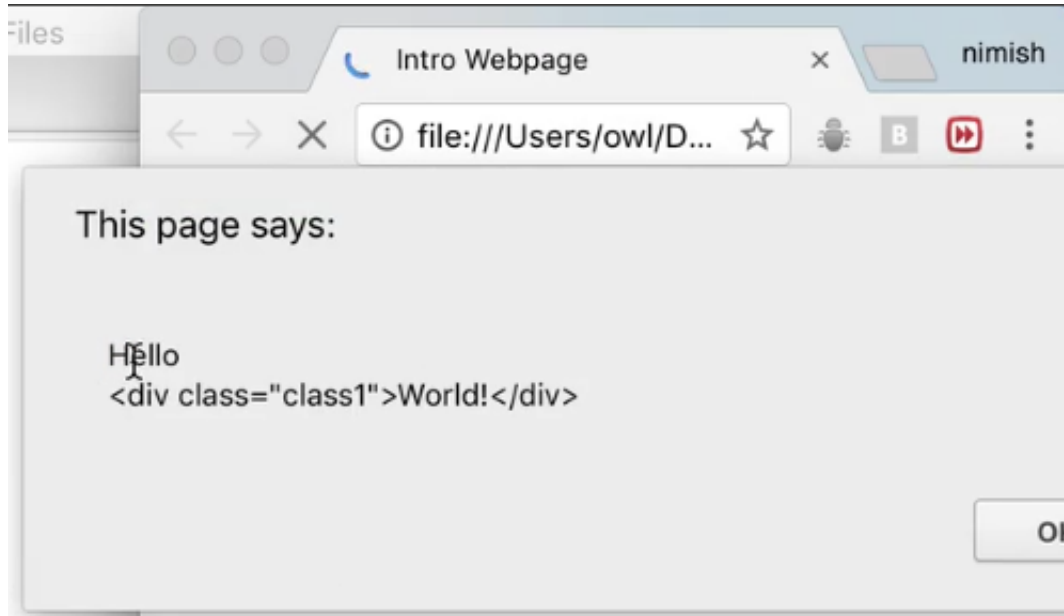
Now we want the **class**, but let's pretend we don't actually know the **class name** for this **div**. Using **getAttribute**, we can get and save a specific **attribute**, whether **class**, **ID**, or something else.

```
var div1Class = div1.getAttribute('class');
```

Remember from last lesson that we can also get the **inner HTML** as an attribute. Keep in mind, however, that it will literally select everything within the targeted tags. In other words, if we replace

the **inner HTML**, we're replacing **everything** between the tags, even if it's more HTML.

```
var div1HTML = div1.innerHTML;  
alert(div1.innerHTML);
```



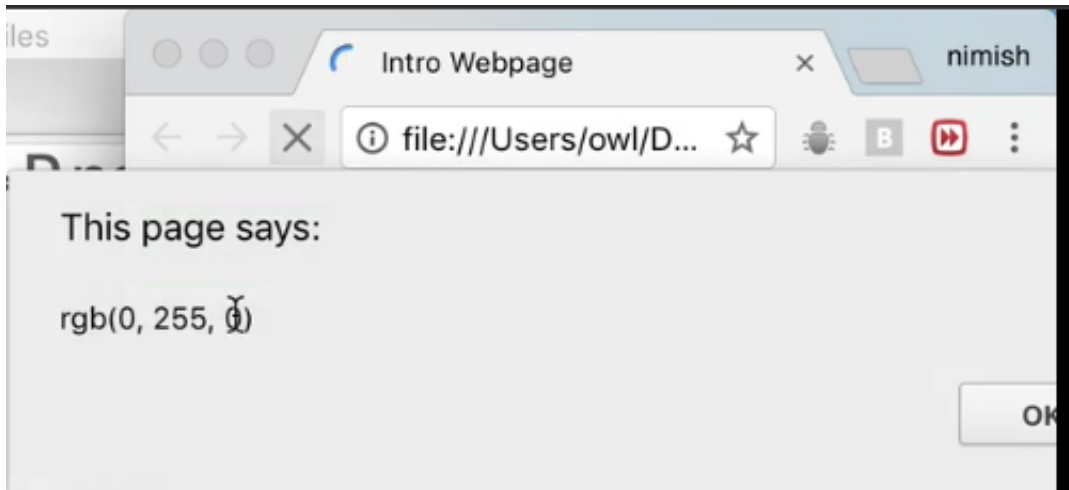
What if we wanted to get a particular **style**, though? While we could use **getAttribute** to find **style**, this would result in a very long list since it would get **all** the style attributes. Instead, using our **div1** variable, we can use **.style** to find a particular style attribute. In this case, we can locate **background color**.

```
var noValueDiv1Color = div1.style.backgroundColor;
```

(**Note:** We can't use hyphens in Javascript, so be sure to check your formatting for these attribute names).

If you try to use an **alert** on the above, though, you won't see anything. This is because our **script runs before the rest of the page**. This means none of our styling exists before this script runs. There are a few ways around this, one which we'll save for later. One way we can get around this is to use **window.getComputedStyle**. This function forces the style to implement before the script runs so that we are able to use it for things like an **alert**.

```
var div1ComputedStyle = window.getComputedStyle(div1).backgroundColor  
alert(div1ComputedStyle);
```



Setting Attributes

Now that we can retrieve attributes, let's learn how to set them. One thing we can change is the **class** and **ID** names. While not advised, it is quite easy using the **setAttribute function**. With this function, we can declare the **type** of attribute we want to set followed by the name. In the code below, we will change our **div1 ID div** to have **class2** as its **class**.

```
div1.setAttribute('class', 'class2');
```

Given that we can retrieve it, we can use **innerHTML** to also set our **HTML**. Again, remember that this will replace **all** the **HTML** that is inside the specified tags. In this case, our entire "Hello World!" is replaced by "Goodbye".

```
div1.innerHTML = 'Goodbye';
```



Of course, we can also set our **styles**. Like above, all we need to do is use an **equal sign** to assign a different value.

```
div1.style.backgroundColor = '#0000ff';
```



However, the above method is best for setting **one style** at a time. If we need to set multiple **style attributes** all at once, we can use **setAttribute** and use **style** as the argument to set a variety of them all at once.

```
div1.setAttribute('style', 'height: 100px;');
```

Note that by using **setAttribute** we overwrite **all** the style that exists for whatever element we're targeting. Thus, if we used the two previous codes in tandem, we'd overwrite our new blue background color so it wouldn't exist. Yet because of the page load process the green is loaded **after the script**, thus it would still appear.

Play around with the skills you've learned in this lesson until you feel comfortable using them. In the next lesson, we'll learn how to add **HTML nodes** using just **Javascript**.

In this lesson, we're going to learn how to add and remove **HTML nodes/elements** to our **DOM** tree using **Javascript**. Again, it may be helpful to **comment out** the **Javascript** from last lesson in our "Intro Webpage" work file.

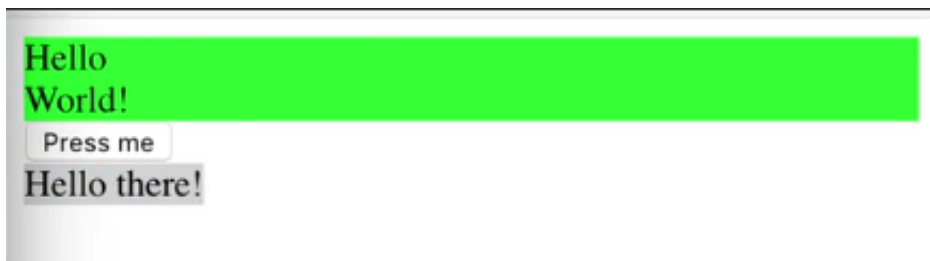
Creating a New HTML Node

For this example, we're going to add in a **new div** to our **HTML**. In the **script tags**, we will first use **createElement** to create a **div**, and then we'll set the **new divs inner HTML** to be "Hello there".

```
var newDiv = document.createElement('div');  
newDiv.innerHTML = 'Hello there!';
```

Now to add it to our page, we can use **appendChild**. We can add this virtually anywhere we want. For instance, with the following code, we add it at the bottom of our **document body** so it appears below the button.

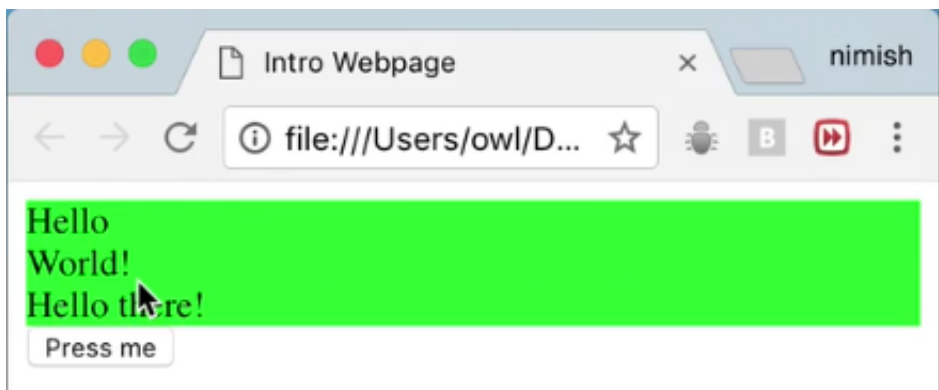
```
document.body.appendChild(newDiv);
```



However, we can also combine it with previous lessons and add it in a specific place. In this case, we will **find** the **element** with the **ID** of **div1**, and then use **appendChild** on that **HTML element**.

```
var div1 = document.getElementById('div1');  
var newDiv = document.createElement('div');  
newDiv.innerHTML = 'Hello there!';  
div1.appendChild(newDiv);
```

Now you can see that it appears **at the end** of our **div1 element**.



Removing an HTML Node

Let's say we want to remove the **div1 div** all together from our **document**. To do this, we can simply use the **removeChild function** which allows us to specify which element to remove.

```
document.body.removeChild(div1);
```

Notice that for these examples, we need to **traverse** our code from the **document** to the **body** in order to add and remove elements from the main content. If we didn't specify the **body** (or similar preexisting tag like **head**), we'd have to add or remove a **tag** as a **direct child** to **document** which **isn't allowed**.

Knowing this method can be useful when working with a large amount of dynamic data, but a future lesson will cover **showing** and **hiding** the **nodes** instead.

In the next lesson, we'll learn about adding **event listeners** using **Javascript**.

In this lesson, we're going to learn how to add **event listeners** to our **DOM items** using **Javascript**. As usual, you should **comment out** the **script code** from the last lesson in our "Intro Webpage" project.

Basic Event Listener

To add an **event listener**, we need to first get our **element** that we want to add it to. Since our **button** makes the most logical sense, we'll **get that element** by its **ID**.

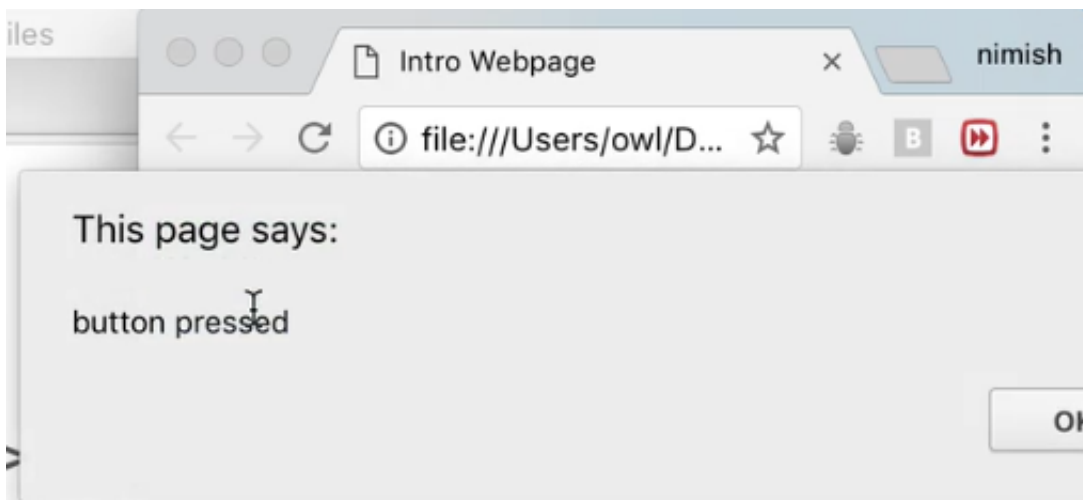
```
var button1 = document.getElementById('button1');
```

Now we can add our **event Listener** by calling the **element name** (button 1, in this case) and using the **addEventListener** function. This particular function takes in **two arguments**: the **type of event** to listen for, and a **function** that it should performed when the **event** is triggered.

```
button1.addEventListener('', function(event){  
  
});
```

There are a variety of **events** we can use for **type of event**, such as **click**, **keyup**, and **keydown** to name a few. Since we have a button, we'll be listening for a **click** in this case. As for the **function**, we'll print a simple **alert** that tells us the button was pressed.

```
var button1 = document.getElementById('button1');  
button1.addEventListener('click', function(event){  
    alert('button pressed');  
});
```

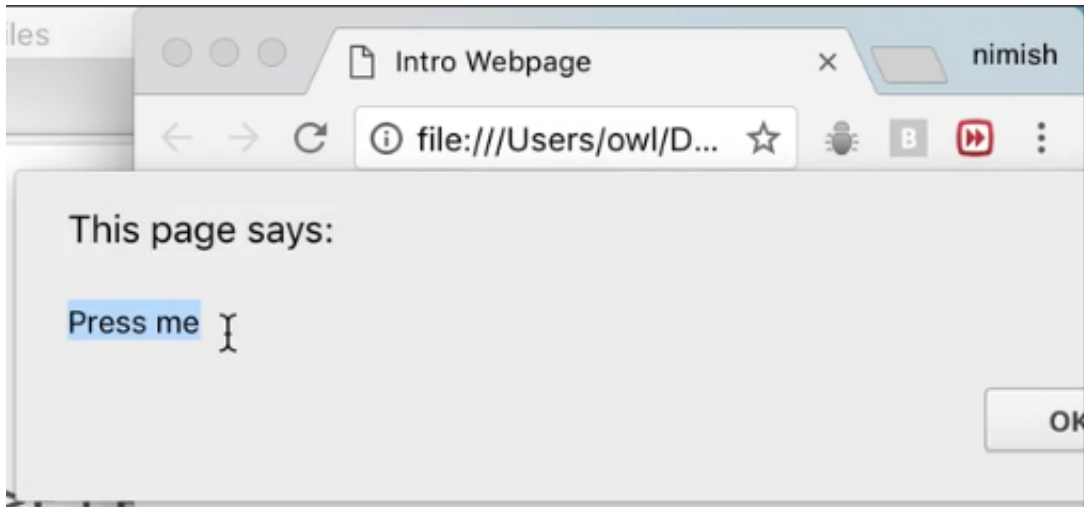


(**Note:** By **Googling** for a list of **Javascript events**, you can find many more event types at your disposal.)

Using the Event

Working with the **event** itself (that was passed into the function) is also helpful. Using **target**, we can have our **eventListener** determine which **page element triggered** the **event**. This is useful for instances when you have multiple elements using the same **event listener** but also need to know which of those elements triggered the event in the first place. For our example, we'll use an **alert** to show the **innerHTML** of the button.

```
var button1 = document.getElementById('button1');
button1.addEventListener('click', function(event){
    target = event.target
    alert(target.innerHTML);
});
```



Page Load Event

In previous lessons, we talked quite a lot about how the **script runs before the rest of the page loads**. When the page does load, however, an **event** is automatically triggered. By adding a **listener** to our **document**, we can listen for this **DOMContentLoaded event** to find out when that is.

```
document.addEventListener('DOMContentLoaded', function(event){
    alert('Page has been loaded');
});
```

Keep in mind this does not mean the **window** itself has loaded, as this is a separate **event** we'd have to listen for.

MouseOver Event

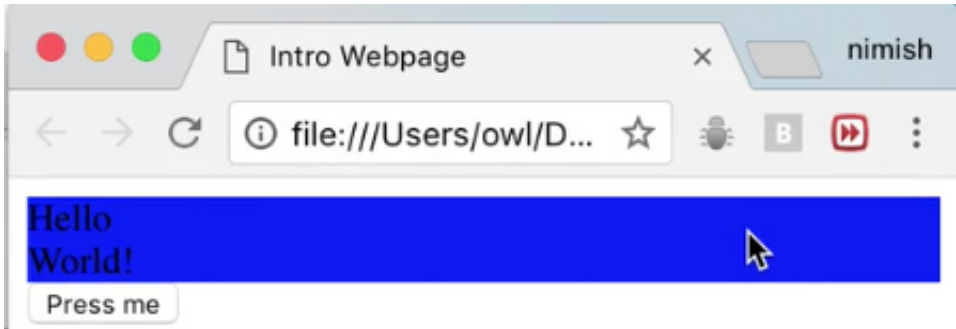
One last **event type** we're going to demonstrate is **mouseover**, which triggers when the mouse hovers over the **DOM element**. For our example, we'll use **div1** as the target for our **event listener**. Unlike before, though, we won't save a separate **variable** that contains our selection for **div1**. Instead, we will use a single line and chain all our elements together.

```
document.getElementById('div1').addEventListener('mouseover', function(event) {
    event.target.style.backgroundColor = '0000ff';
});
```



```
});
```

As you can see with this **listener's function**, we get the **target** of the **event** (div1) and change the **background color** to blue. However, please note that without another **event listener**, the color stays blue and doesn't change back to green. Additionally, our **hover** goes one **div** at a time, so we can change "World!" separately from "Hello".



In the next lesson, we'll learn how to set up a client-side form and manipulate that with **Javascript**.



In this lesson, we're going to look at **client side forms** and using **Javascript** for them in our "Intro Webpage" file. Covering the **server side** portion involves **PHP**, which is beyond the scope of this course.

Creating a Simple Form

Under our "Press me" button, we're going to set up a simple form that has a **text input** for a name and an **input** button to **submit** it. So we can use **event listeners**, we'll give our form a unique **ID**.

```
<form id='form'>
  Name: <input type='text'>
  <input type='submit' value='Save'>
</form>
```

Keep in mind the above is not the only way you could set up your submit button. For example, by having the **type** be a **button** and adding an **onclick** to the tags, we could then specify the function we want called. For our purposes, however, we're going to stick with the **submit type**.

In the **script tags**, the first thing we're going to do is **get** our **form** by its **ID**.

```
var form = document.getElementById('form');
```

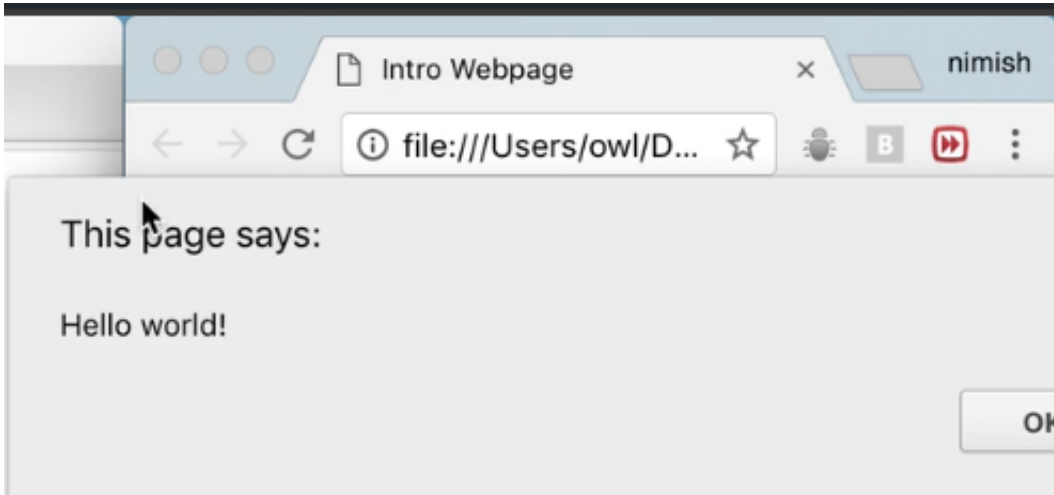
Now we can add the **event listener** to our form. In this case, we'll be listening for a **submit event**. As for the **function**, we want to print a simple **alert** of our **text input**. Fortunately, by using **form.elements**, we can use an **array index** to access the input field we want (0 in this case for the first element).

```
form.addEventListener('submit', function(event){
  var textInput = form.elements[0]
  alert(textInput.value)
});
```

However, our code is missing something. If we were to use our **submit button** now, the page would refresh and everything would be cleared. We don't want this to happen, so we need to tell our **event** to **prevent** this **default** behavior.

```
form.addEventListener('submit', function(event){
  event.preventDefault()
  var textInput = form.elements[0]
  alert(textInput.value)
});
```

Now our code will work without clearing the text field.



Note that **preventing default** is not advised for a **server side form**, but it is fine for a **client side form**.

Intro Section Summary

Over the past few lessons, we've not only learned how to access **DOM elements** by **class**, **ID**, **tag**, and **querySelector**, but how to change their attributes dynamically. We've also learned how to use **event listeners** to add extra functionality to our pages.

In the next set of lessons, we'll be moving into the projects demoed in the very first lesson, with the first one being our Spanish flashcards.



In the next four lessons, we're going to create our first project which consists of interactive flash cards. These cards will feature a picture, English word, and button that allows the user to show or hide the answer for the Spanish equivalent. Throughout these lessons, we'll work with **event listeners** and learn how to **show** and **hide elements** on the **DOM**.

Before we move on, be sure to create a **new project folder** called **Spanish Flashcards Code File**. Additionally, create a **new HTML file** called **Flashcards Page** in **ATOM** (or other code editor) and save it into this project folder. You may also find it helpful to open up your browser to test the file as we go.

In the next lesson, we'll begin writing the **HTML skeleton** for our project.

In this lesson, we're going to build the **HTML backbone** for our "Flashcards Page". Double-check that your file is indeed an **HTML** file before we begin.

HTML Skeleton and First Card

Like with any webpage, we'll first begin with the **head** and **body tags**. In the **body tags**, we'll also set up a **div container**. This will allow us to do things like changing the overall position of all the cards if we need to.

```
<head></head>
<body>
  <div class='container'>

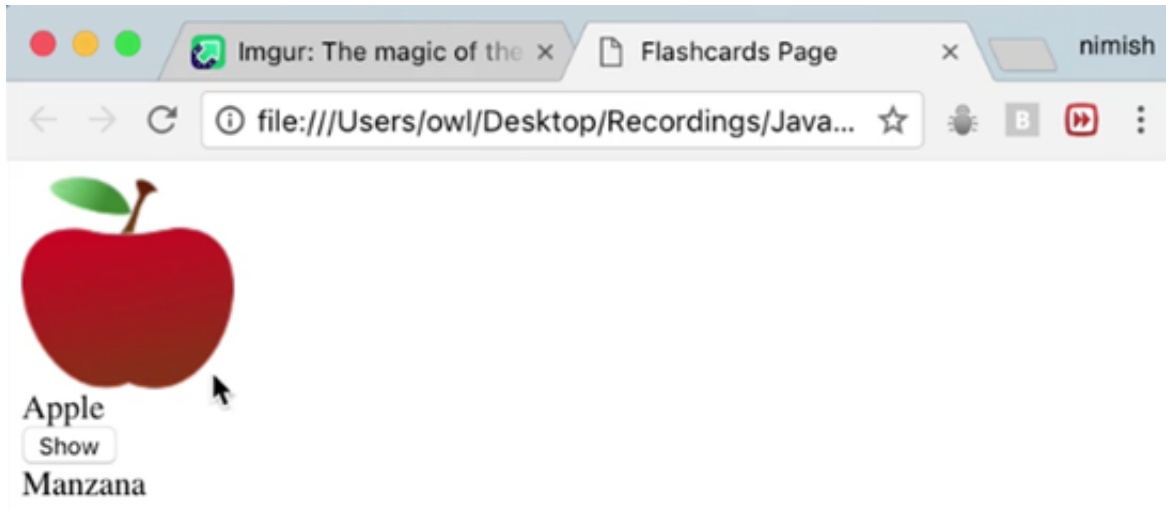
    </div>
</body>
```

In our **div container**, we can start creating our individual cards. For each **card**, we'll contain them in their own **div** with a **class** of **card**. As mentioned last lesson, each of these cards will contain a picture, English word, and show/hide button. However, with each of these cards, we also need to include the **Spanish word answer** in the **HTML**. When we get to the **styling**, we will set it up so that the page loads with these elements hidden.

The first card we'll create is for an "Apple". Since we're not packaging this as a web page, for the image we'll just link to one directly on **Imgur** (<https://imgur.com/>). However, you can also find the images we'll be using in the **Course Files' Images** folder for this project. Besides the image, each of our components needs their own **class** so that we can call upon them. We will use **card-title**, **card-button**, and **card-answer**.

```
<div class='card'>
  <div></div>
  <div class='card-title'>Apple</div>
  <button class='card-button'>Show</button>
  <div class='card-answer'>Manzana</div>
</div>
```

It's nothing fancy yet, but we can see our card so far.

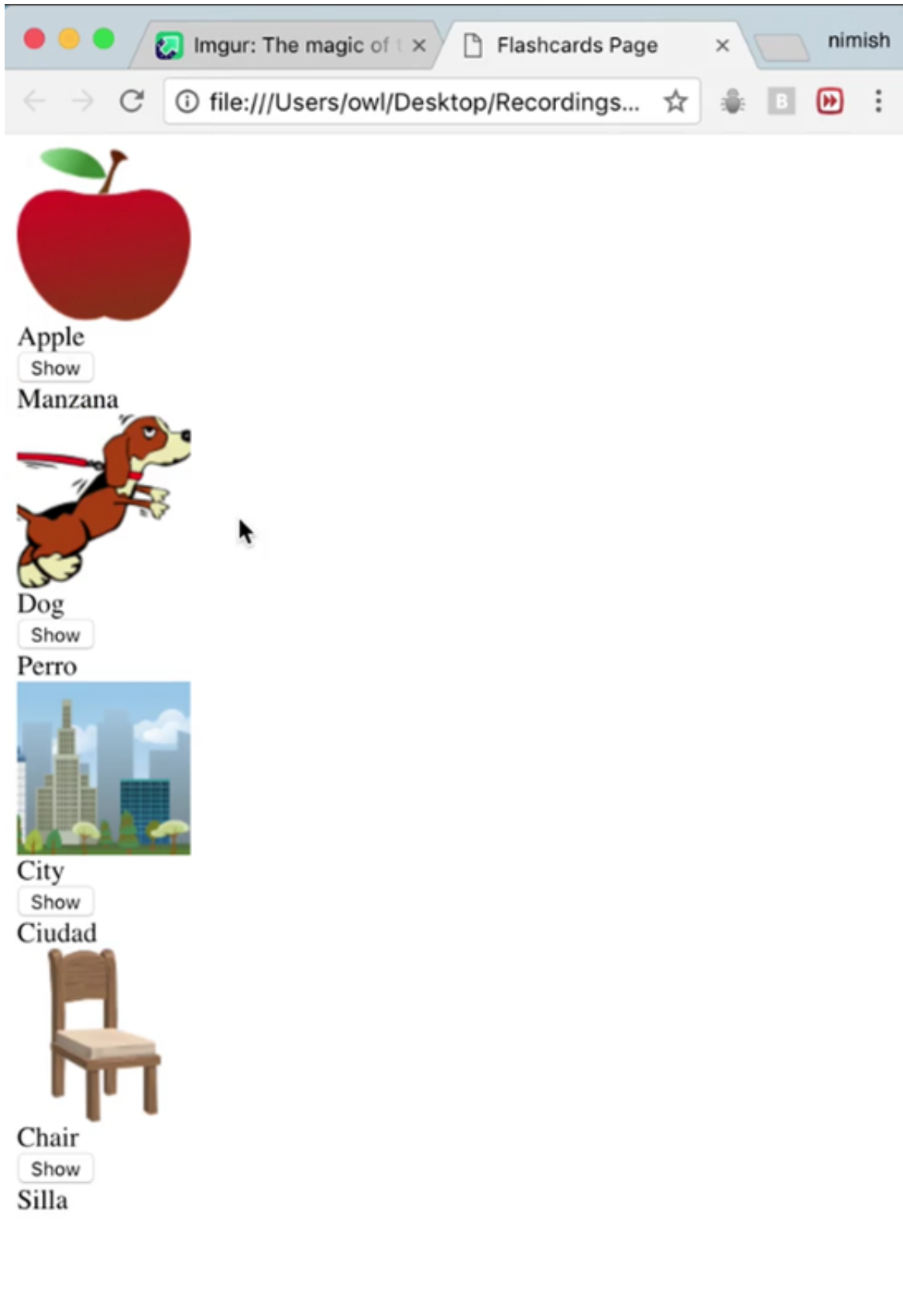


Other Cards

Now that we've created our first card, we can **copy** and **paste** this code to create three more cards for dog, city, and chair. Of course, we need to change the relevant English word, Spanish word, and image to reflect the appropriate object. Again, we'll just be linking to images on **Imgur**.

```
<div class='card'>
  <div></div>
  <div class='card-title'>Dog</div>
  <button class='card-button'>Show</button>
  <div class='card-answer'>Perro</div>
</div>
<div class='card'>
  <div></div>
  <div class='card-title'>City</div>
  <button class='card-button'>Show</button>
  <div class='card-answer'>Ciudad</div>
</div>
<div class='card'>
  <div></div>
  <div class='card-title'>Chair</div>
  <button class='card-button'>Show</button>
  <div class='card-answer'>Silla</div>
</div>
```

Save the file and test it in your browser. Everything should be looking decent.



In the next lesson, we'll style the page with **CSS** so that our cards are better looking.



In this lesson, we're going to add some **CSS styling** to our "Flashcards Page" file. This should be fairly straight forward since we assigned **classes** to effect similar elements in all the cards.

Styling our Flashcards

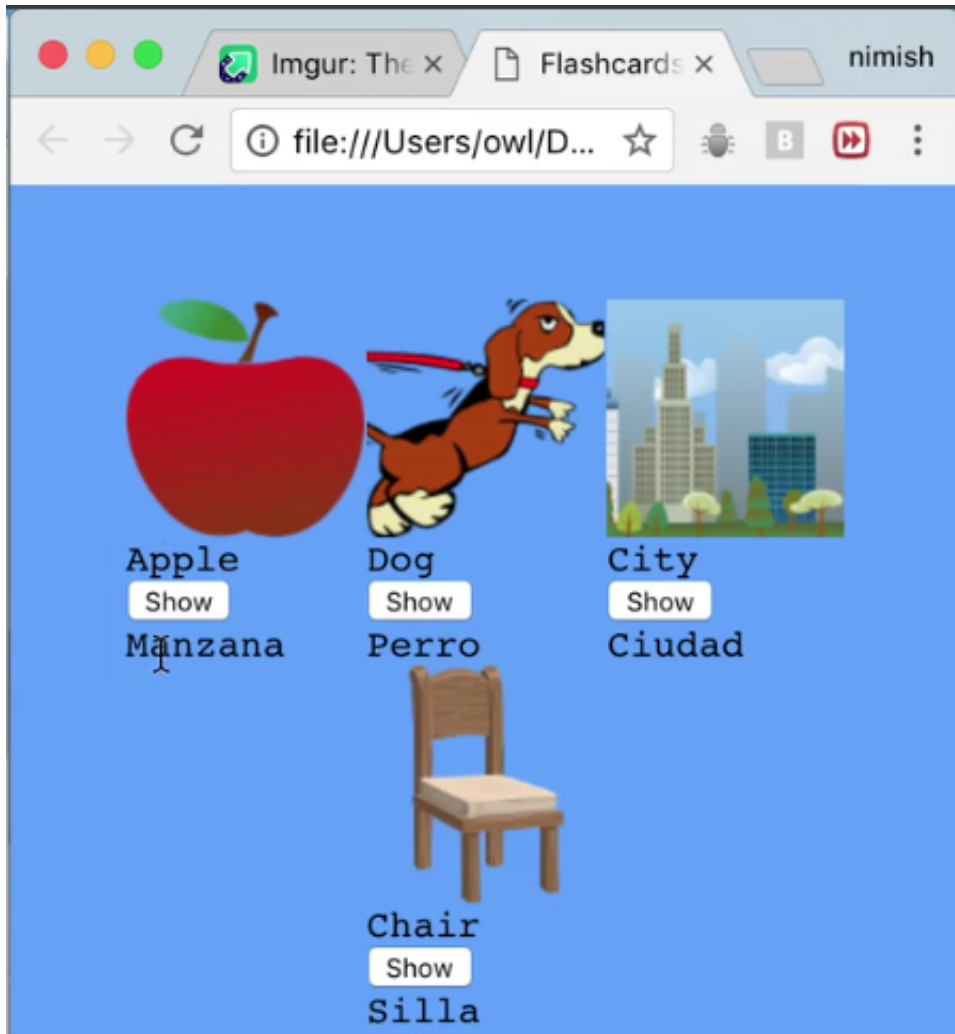
Up in the **head tag**, we're going to add a **style tag** in which we can add our **CSS**.

```
<head>
  <style>

  </style>
</head>
```

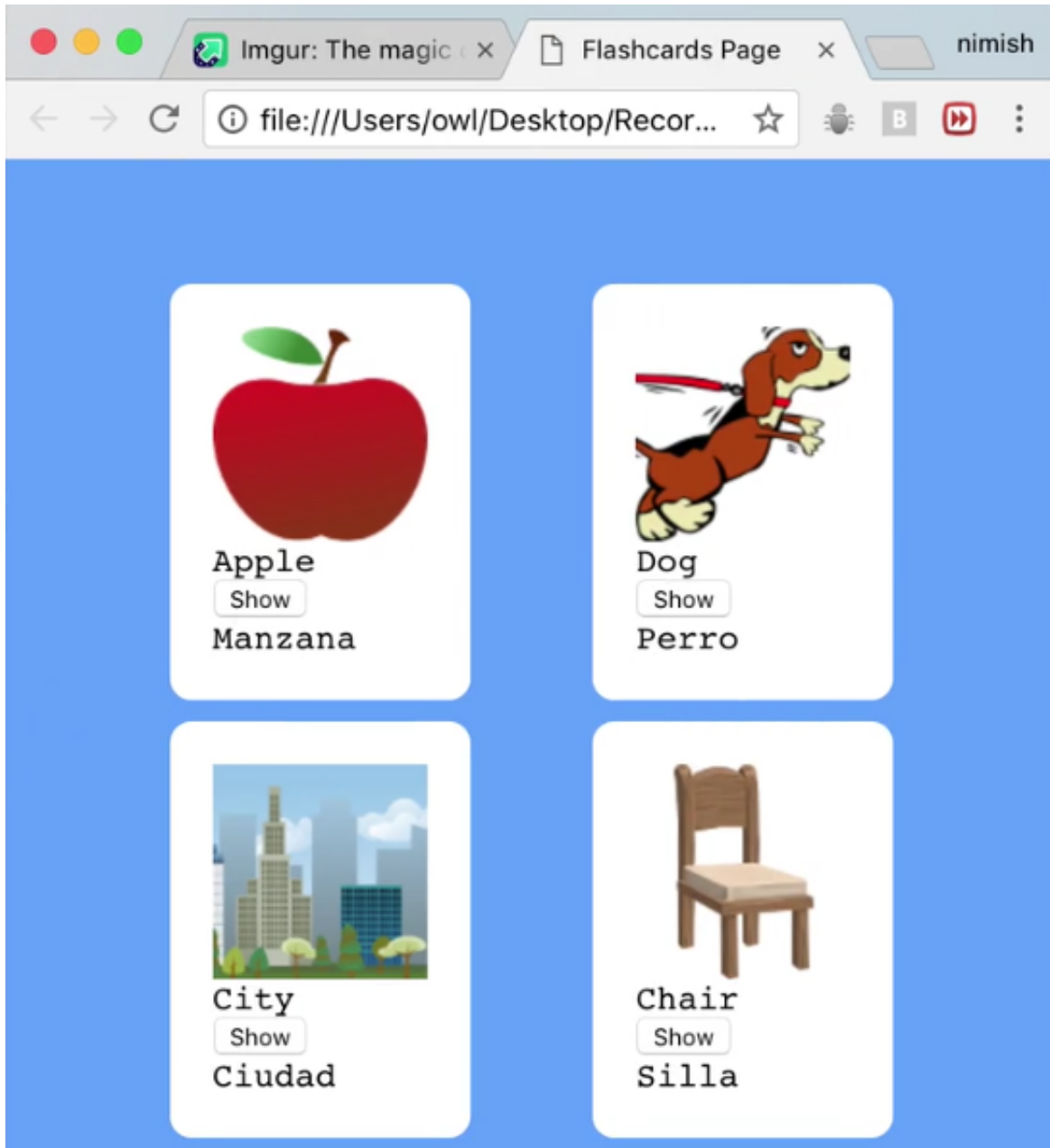
We'll begin styling the **body** and **container** first. For our **body**, we'll change the font to **Courier** and give the **background** a nice **blue** color. So that our card's display better on larger screens, we will use **flexbox logic** to **display** everything as a **row** that will **wrap** automatically.

```
body {
  padding: 40px;
  background: #5f9fff;
  font-family: courier;
}
.container {
  display: flex;
  flex-flow: row wrap;
  justify-content: space-around;
}
```

We can now move on to styling each card. We will add various **margin** and **padding** attributes that you can play around with to your liking. Primarily, though, we want to make the cards **white** and have a slight curve with **border-radius**.

```
.card {  
  flex: none;  
  margin-top: 10px;  
  padding: 20px;  
  border-radius: 10px;  
  background: white;  
}
```



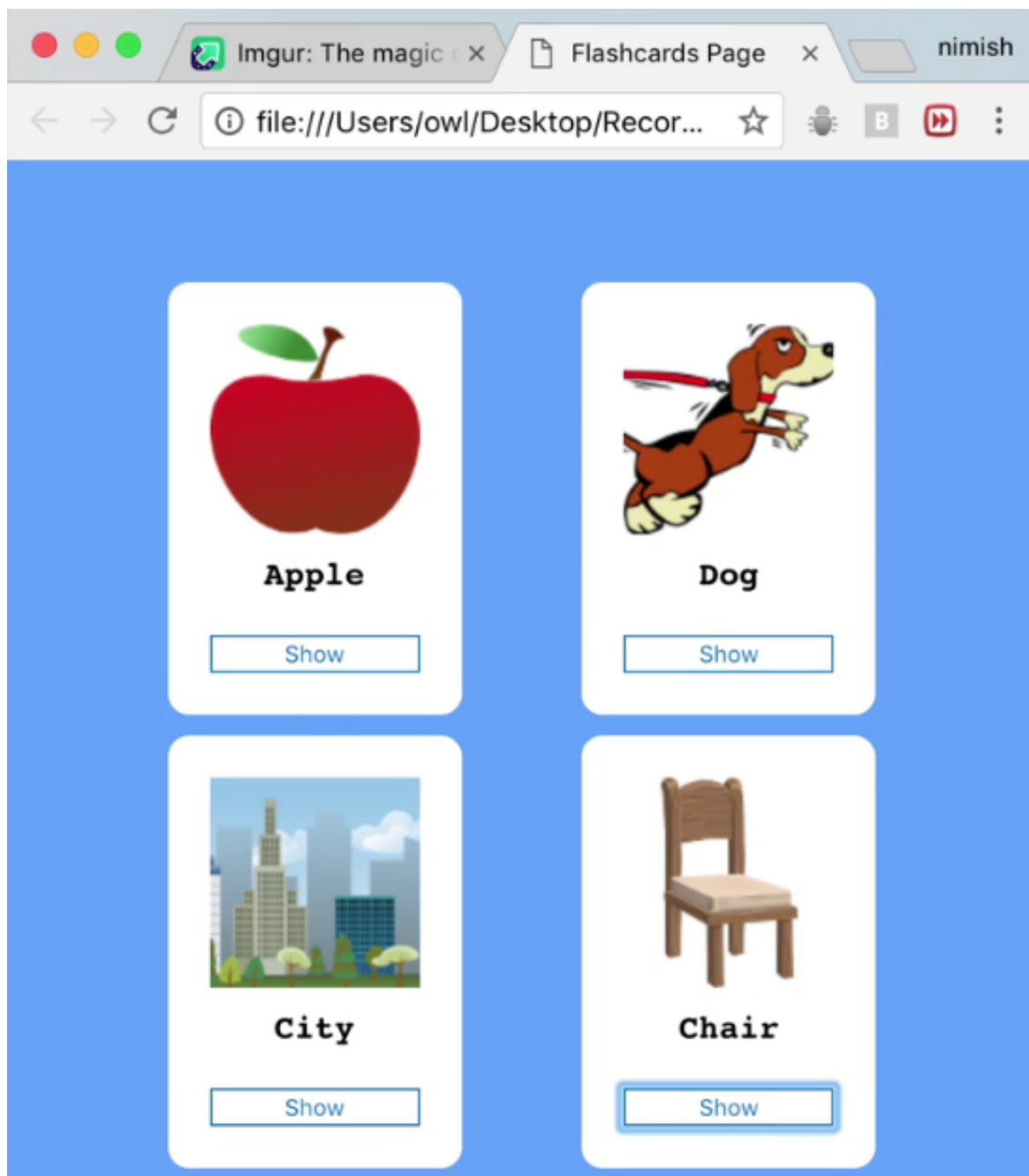
Next for the **title** and **answer text**, we want to make sure the **font weight** is **bold** and the text is **centered**, as well as add some margins to the top and bottom. Meanwhile, we will change the **button** significantly by making it **white**, giving it a **blue border**, changing the text to **blue**, and changing the **width**.

```
.card-title, .card-answer {  
  margin-top: 10px;  
  margin-bottom: 10px;  
  text-align: center;  
  font-weight: bold;  
}  
.card-button {  
  width: 100%;  
  margin-top: 10px;  
  border: 1px solid #0070bb;  
  background: white;
```

```
color: #0070bb;  
}
```

Last but not least, we want our **card answer** to not be **displayed** when the page loads. This is as easy as specifying the **display** as **none**.

```
.card-answer {  
  display: none;  
}
```





In the next lesson, we'll work on the **show/hide** functionality for just one card.



In this lesson, we're going to start implementing the **JavaScript** that will allow us to **show** and **hide** our answers. For this lesson, we're going to show how to do just one card, since multiple cards need a more robust implementation to work.

Showing the Answer

Above the **closing body tag**, add a **script tag** so that we can begin coding our script.

```
<!--Rest of the code-->
  <script>

  </script>
</body>
```

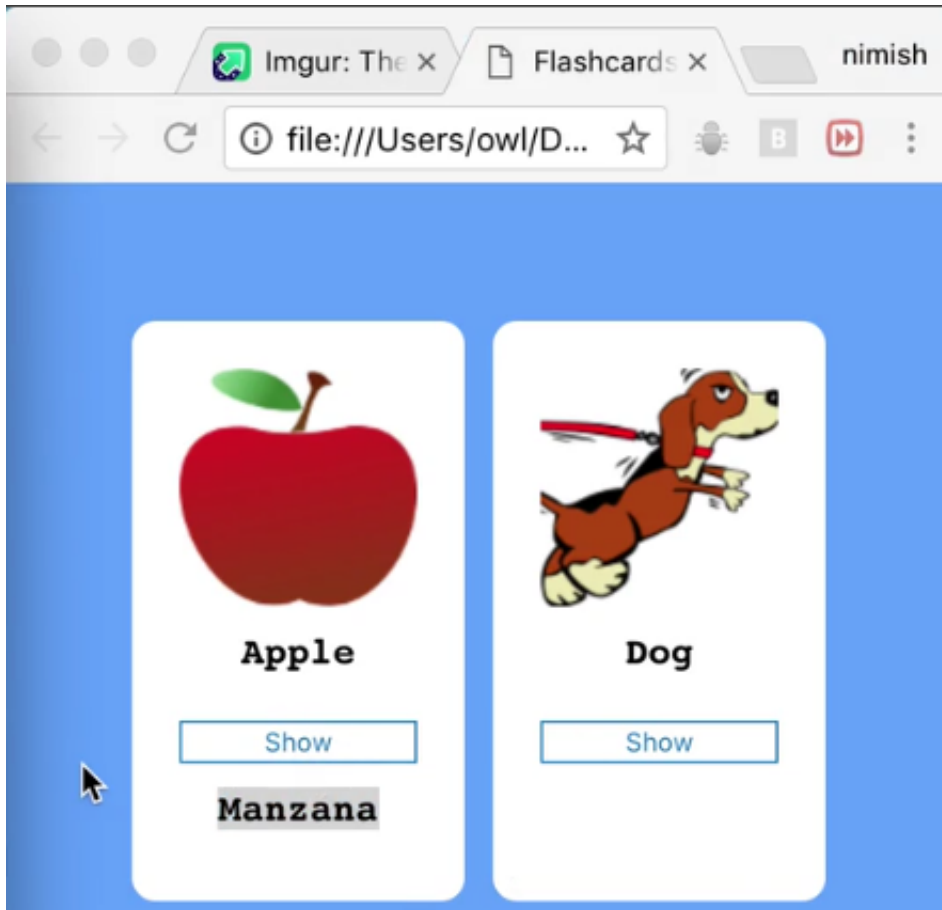
Let's focus only on the first flash card. To add interactivity, we need to access the **button** and **answer** on our **DOM**. We can use **getElementsByClassName** to do so since they both have **classes** we can find. However, remember this returns an **array**, so on these lines we need to specify that we only want the first element which is at **index 0**.

```
var firstButton = document.getElementsByClassName('card-button')[0];
var firstAnswer = document.getElementsByClassName('card-answer')[0];
```

Now that we can access the elements, we can add an **event listener** to our **button** that is listening for a **click**. For the first part of our function, we want to change the **answer's style** to **display block**, which you may remember from an earlier lesson when we used the technique to change the background color.

```
firstButton.addEventListener('click', function(event) {
  firstAnswer.style.display = 'Block';
})
```

Our answer should **show** now, but we can't yet **hide** the **answer** and our button text doesn't change either.



Show and Hide Toggling

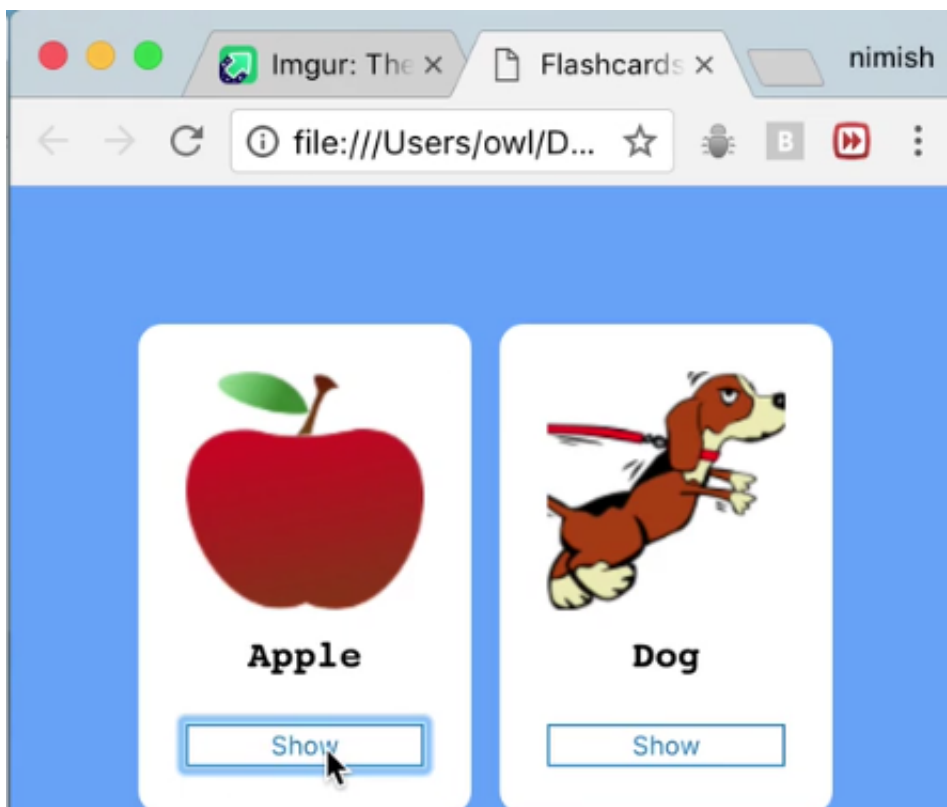
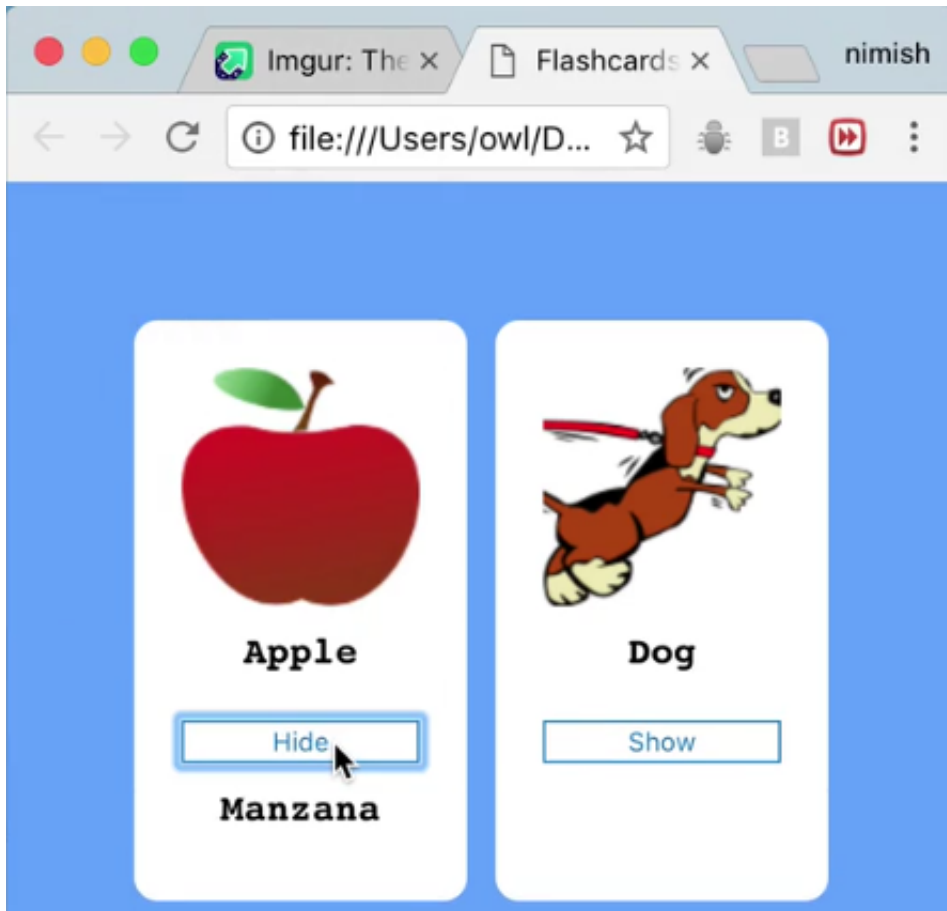
In order for us to be able to toggle between the **show** and **hide** abilities of our answer, we need to use an **if/else if statement**. For our conditions, we want to check the **button text**. If the button text says **“Show”**, we want the answer to **display** as **block** when clicked. If the text says **“Hide”**, we want the answer to **display** as **none** when clicked.

By using **innerHTML**, we can check within the first button to not only see what the text is, but to also change it. This allows us to **change the text** to **“Hide”** when the answer is set to **display block** and **“Show”** when the answer is set to **display none**. This will result in the intended toggle functionality.

```
firstButton.addEventListener('click', function(event) {  
  if (firstButton.innerHTML == 'Show') {  
    firstAnswer.style.display = 'Block';  
    firstButton.innerHTML = 'Hide';  
  } else if (firstButton.innerHTML == 'Hide') {  
    firstAnswer.style.display = 'None';  
    firstButton.innerHTML = 'Show';  
  }  
})
```

(Note: You could also make your conditions check for the display instead of the button text. Remember, there are numerous ways to get the same functionality in coding, so find one that works best for your situation.)

Save the file and you can test out the functionality.





Now that we understand a little how **showing** and **hiding** works, in the next lesson we're going to add this functionality to all the cards. You can challenge yourself by trying to implement it using a **for loop**. In the next lesson, we will reveal the answer to this challenge and show you how to do it.



In this lesson, we're going to apply the knowledge and logic we used last lesson so that we can apply it to all the cards on our "Flashcards Page". The method we learn in this lesson will work not only for our present cards, but any future cards we might add.

Getting All the Cards

To begin this lesson, **comment out** all the **Javascript code** we wrote last lesson so it doesn't interfere with our current code. Now, in the **script tags**, we will first get a reference to all our cards using the **card class name**.

```
var cards = document.getElementsByClassName('card');
```

Now that we have an **array** of all our **cards**, we can use a **for loop** to **iterate** through each card. Using each **index** that we find we can access each specific card that is stored there. Then, with a **querySelector**, we can find the **button** that belongs to each card with that **index** and save it in a variable. Last but not least, we can then add a **click event listener** to these saved buttons.

```
for (var cardIndex in cards) {  
  var button = cards[cardIndex].querySelector('.card-button');  
  button.addEventListener('click', function(event){  
  
  })  
}
```

Event Listener Code

We can add the rest of our code now into the **event listener**. In order to get the functionality we want, we need to know two items: the **button** that **was clicked** and the **answer** it relates to. Unfortunately, it isn't as simple as before because the **event listener** doesn't know which card to target without help. First, to make this work, we need to find out which **button** triggered the **event** using **event.target**.

```
var currentButton = event.target;
```

Now that we know which **button** triggered the event, we can **traverse** the code to get to the **answer**. First, we need to get to the **parent node** (i.e. the **div** higher in the hierarchy) of the button, which would be the **div** with the **card class**. From this **parent node**, we can now see the other **child elements**. From there, all we need to do is use a **querySelector** to find that specific **node's answer** via the **class**.

```
var currentCard = currentButton.parentNode;  
var currentAnswer = currentCard.querySelector('.card-answer');
```

All that's left to do is implement our **show** and **hide functionality**. For the most part, this code will be the same as last lesson, so you can even **copy** and **paste** both the **if** and **else if statements** if you want. However, in this case, we need to use the variables we created in the previous lines. In other words, to access the **answer** we need to use **currentAnswer** to change that **answer's**

display. To access the **button**, we need to use **currentButton**.

```
if (currentButton.innerHTML == 'Show') {  
    currentAnswer.style.display = 'Block';  
    currentButton.innerHTML = 'Hide';  
} else if (currentButton.innerHTML == 'Hide') {  
    currentAnswer.style.display = 'None';  
    currentButton.innerHTML = 'Show';  
}
```

You can now **save** the file and test the completed project!





Project Summary

Through this project, we built an **HTML page** of flashcards which we then styled slightly with **CSS**. Using a combination of **event listeners**, **if/else if statements**, and the **display**, we learned how to **show** and **hide elements** with the **click** of a button. Additionally, using a **for loop**, we managed to apply the logic to all the cards and then **traverse** the code via the **event.target** to locate the appropriate elements to change.

In the next lesson and project, we'll work on creating a list which we can then filter through via an input text field.



In the next three lessons, we're going to create our second project which consists of filtering a list. This list will load with a bunch of potential activities, which the user can then filter through in real time using a **text input box**. Again we'll be using **event listeners**, this time ones added onto our **input**.

Before we move on, be sure to create a **new project folder** called **List Filtering Code Files**. Additionally, create a **new HTML file** called **List Filtering Page** in **ATOM** (or other code editor) and save it into this project folder. You may also find it helpful to open up your browser to test the file as we go.

In the next lesson, we'll begin writing the **HTML skeleton** for our project.



In this lesson, we're going to build the **HTML backbone** for our "List Filtering Page" project. The premise for this project is that we're creating a list of local activities that we can filter through.

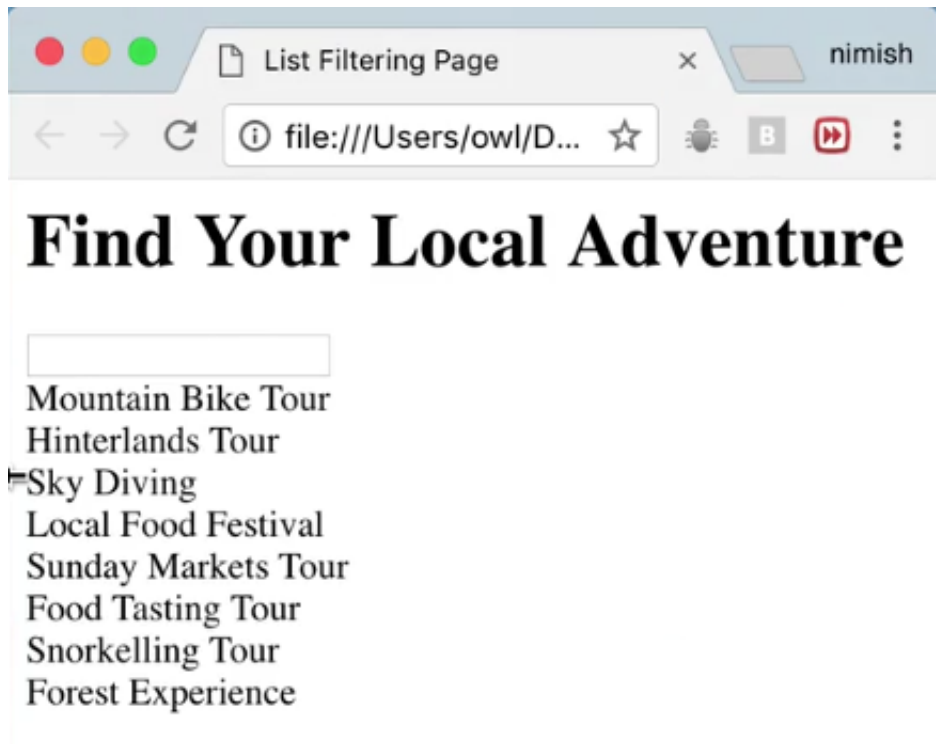
HTML Skeleton

For our **HTML**, we're going to do a simple set up of having a **headline**, **text input area**, and **list**. Our headline will be nothing special. However, with our **text input**, we will store it in a **div container** as well as give the **input** itself a unique **ID**. In order for the filtering to work, we need to make sure our **input field** is left without placeholder text.

In regards to our list, we will also contain these in their own **div container** so we can manipulate all the items as one object later. Each item on our list will be separated into their own **div tags** with a **class** of **adventure**. You can choose whatever adventures you want, but we will be using the ones seen in the code below.

```
<head></head>
<body>
  <h1>Find Your Local Adventure</h1>
  <div class='search-container'><input id='search-bar' type='text'></input></div>
  <div class='adventures-container'>
    <div class='adventure'>Mountain Bike Tour</div>
    <div class='adventure'>Hinterlands Tour</div>
    <div class='adventure'>Sky Diving</div>
    <div class='adventure'>Local Food Festival</div>
    <div class='adventure'>Sunday Markets Tour</div>
    <div class='adventure'>Food Tasting Tour</div>
    <div class='adventure'>Snorkelling Tour</div>
    <div class='adventure'>Forest Experience</div>
  </div>
</body>
```

You can now **save** the file and **double click** it to open it in your browser.



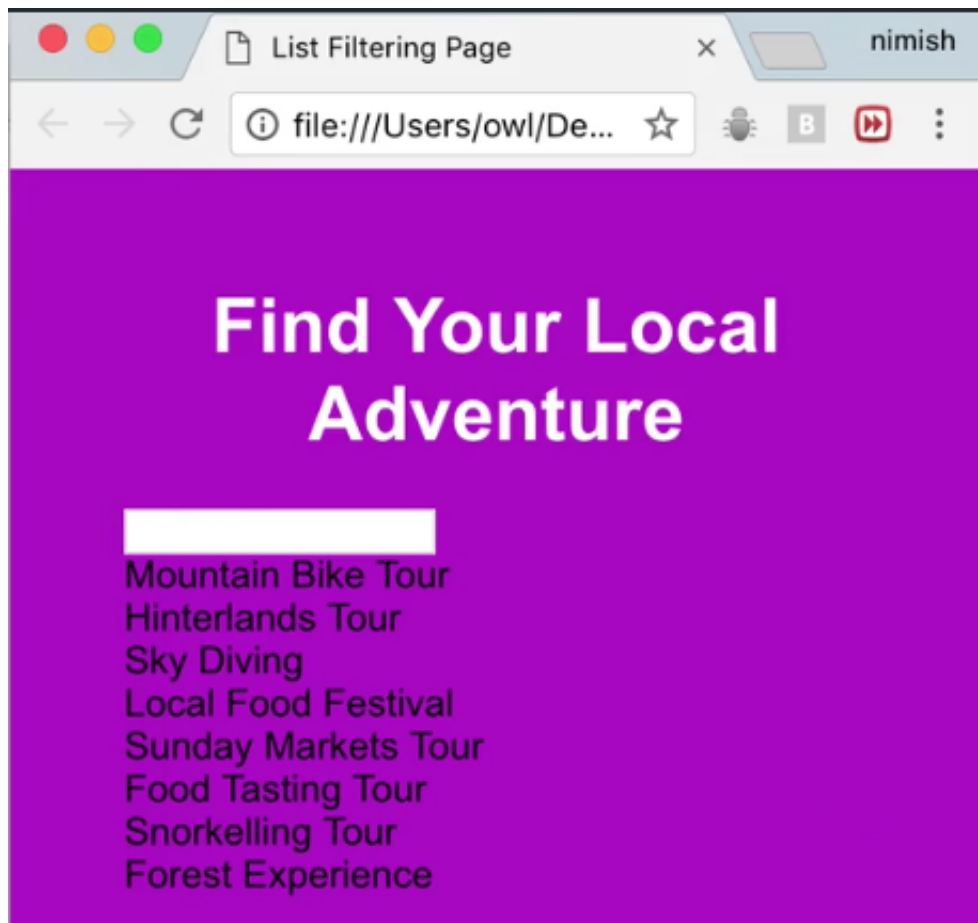
In the next lesson, we'll work on the **CSS styling** for this page.

In this lesson, we're going to work on the **CSS styling** for our "List Filtering Page" project.

Styling the CSS

Once again, we'll be working in the **head tags** where we will use **style tags** to put our **CSS** in. For the **body**, we'll be changing the **background** to be a nice **purple** and changing the **font family** to be **Arial**. We'll also change the padding to give us some more breather room. To match, we'll make our **headline white** and **center** it.

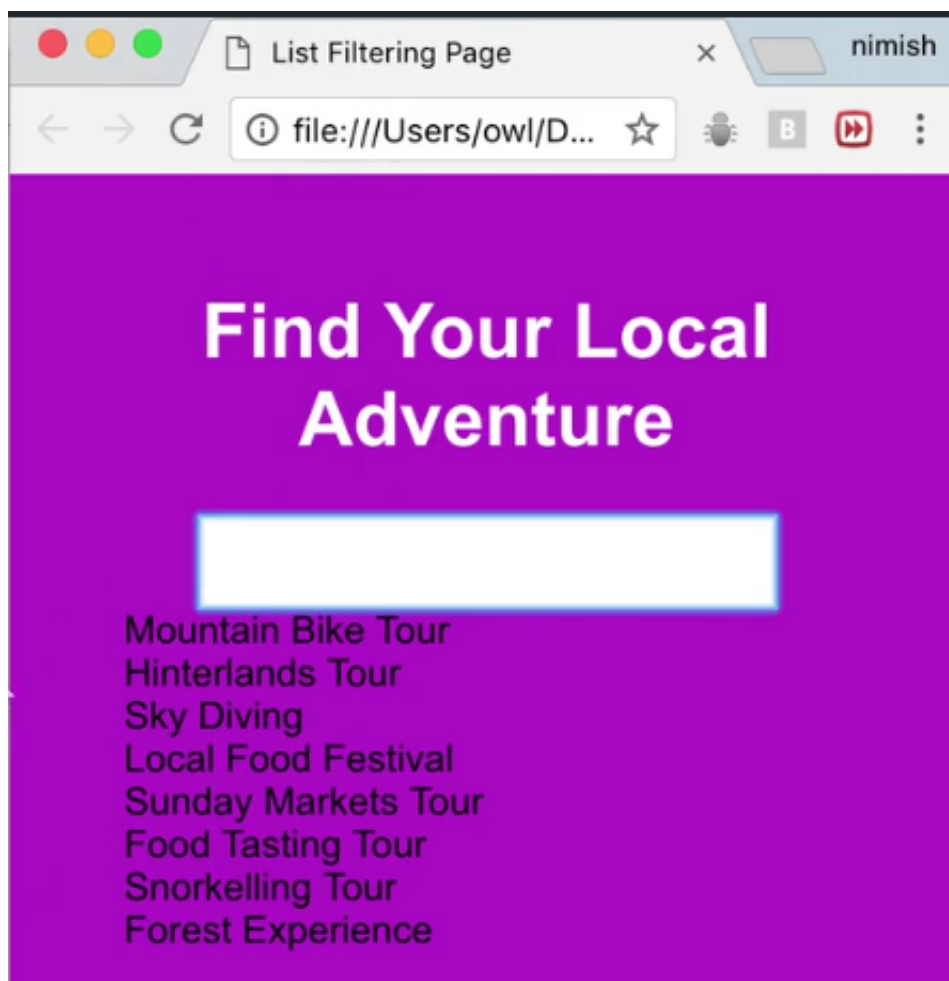
```
<head>
  <style>
    body {
      padding: 40px;
      background: #b10dc9;
      font-family: Arial;
    }
    h1 {
      text-align: center;
      color: white;
    }
  </style>
</head>
```



Now we'll style our **input area**. So that the field will wrap properly and be spaced nicely, we will use

flex display and **space-around** again like we did with the flashcards. We also want to change the size of the **input field**, so using its **ID** we'll change the **width**, **height**, and **font-size**. Make sure to use the **hashtag** to indicate it's an **ID** instead of the **period** used for **class**.

```
.search-container {  
  display: flex;  
  justify-content: space-around;  
}  
#search-bar {  
  width: 80%;  
  height: 40px;  
  font-size: 20px;  
}
```

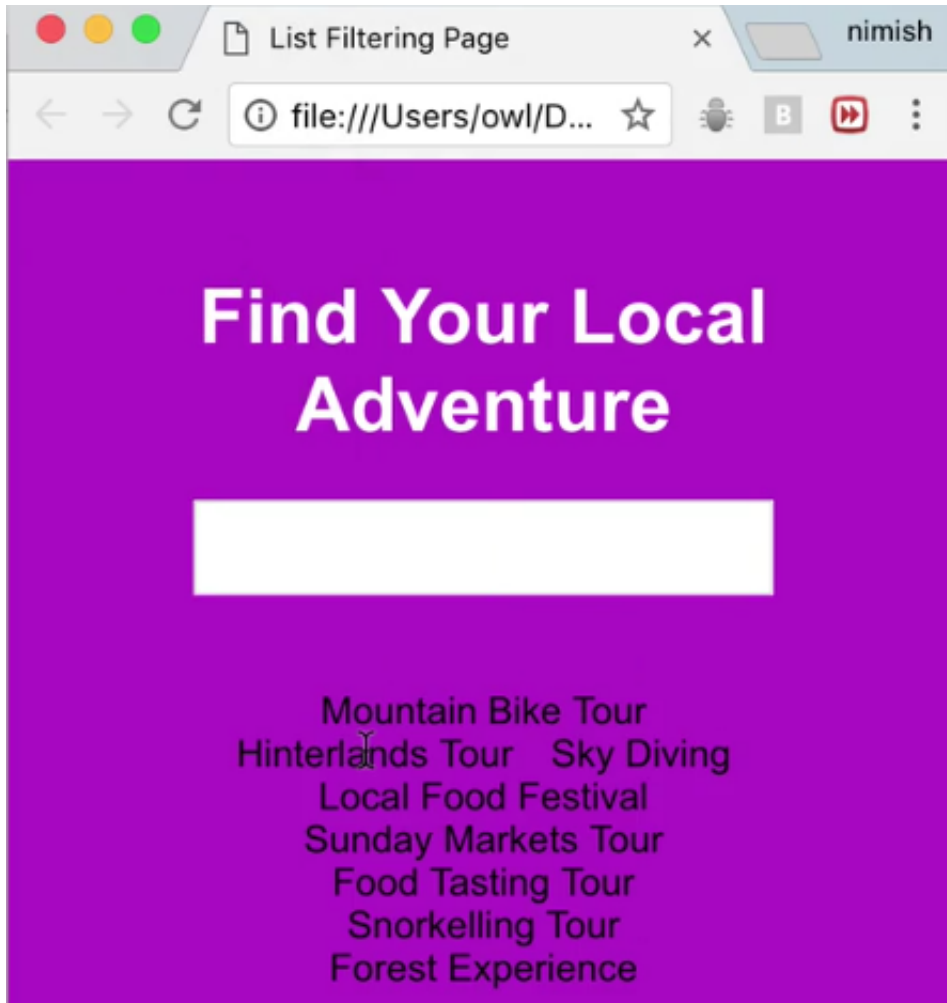


Last, let's style the list. Rather than an up and down look, we want to display as much as we can in a row. Similarly to the flashcards, we can use the **flex display** and **row wrap** technique to achieve this. We will apply this to our **adventures-container class** so it applies to the whole list. Make sure to use a margin so the items aren't packed together.

```
.adventures-container {  
  display: flex;  
  flex-flow: row wrap;  
  justify-content: space-around;
```

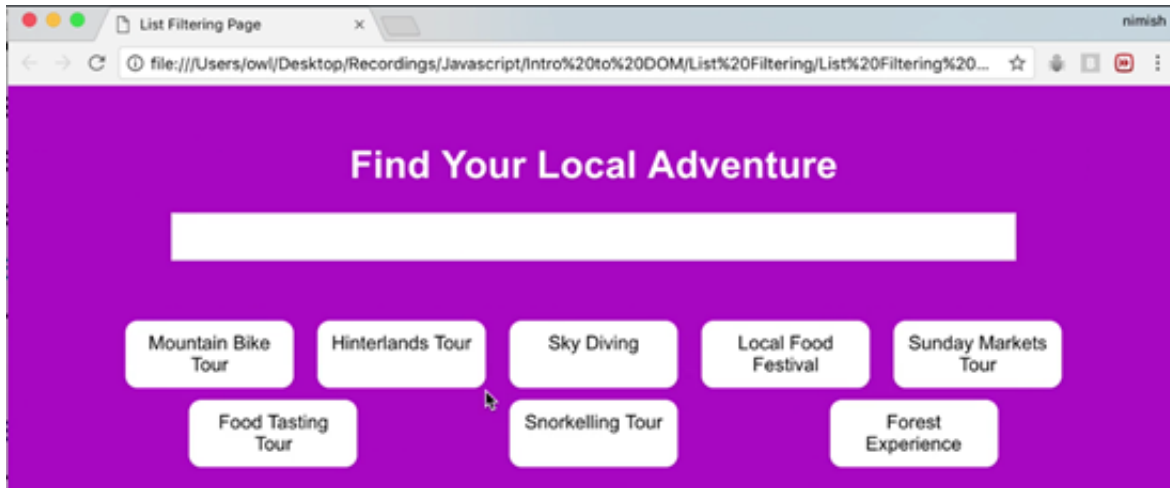


```
} margin: 40px;  
}
```



We also want to give our various list items with the **adventure class** their own little sort of bubble. We can do this setting a **fixed width** and changing the **background** to **white**. Besides padding and margins, we'll also curve these elements with **border-radius** and **center** the text.

```
.adventure {  
  width: 120px;  
  padding: 10px;  
  margin-top: 10px;  
  border-radius: 10px;  
  text-align: center;  
  background-color: white;  
}
```



Our **CSS styling** is done, so next lesson we'll move on to adding the **Javascript functionality** for this project.



In this lesson, we'll be implementing the functionality for our "List Filtering Page" project. For our functionality, we want whatever we type in the input box to filter out list items that don't match.

Making a List to Search

Our first step is to create a **list** of **strings** within our **script tags** (that we'll open up before the **body closes**). This way our page isn't constantly having to look through each item's **HTML** to determine if it's a match in real time. While we could do this manually, if we had a list of hundreds or thousands of items, this wouldn't be practical. Thus, we will learn to do this so our code does all the heavy lifting.

To begin, we first need to create a **variable** to store an **empty list/array**. Then, after, we want to store a **list** of all our list items by using **getElementsByClassName**.

```
var adventureList = [];  
var adventures = document.getElementsByClassName('adventure');
```

However, our **adventures variable array** is a list of all the **HTML** for those elements, and we want just the **strings** so we can add them to the **adventuresList array**. Like we did in the previous project, we can accomplish this task by using a **for loop**. For demonstration purposes, we will use a traditional setup for this **loop**. This involves setting an **initial state** ($i = 0$), checking a **condition** (i is less than the length of our **adventures variable array**), and finally running the inner function before increasing the initialized variable.

```
for (var i = 0; i < adventures.length; i++) {  
  
}
```

Now that we have our loop, we can iterate through each item in the **adventures variable array**. Using our **initial state**, we can specify an **index** and use that to temporarily save that specific list item's **inner HTML**. Then, using **push**, we can send the **inner HTML** to our **adventuresList array** so it's just a list of strings.

```
for (var i = 0; i < adventures.length; i++) {  
    var adventureName = adventures[i].innerHTML;  
    adventureList.push(adventureName);  
}
```

Adding the Search Bar Event Listener

We have a **list** of **strings** to search, so now all that's left is to add an **event listener** to our **search bar**. Using that element's **ID**, we'll first get the search bar in our code. Then we can attach the **event listener** to it. This time, we'll be using the **keyup event** which triggers after a user presses a key *and then* lifts the key back up. This applies to any key, even the backspace.

```
var searchBar = document.getElementById('search-bar');  
searchBar.addEventListener('keyup', function(event){  
  
});
```

In this **event listener**, we can now create our **function**. When the user types in the **input**, its **value**, or text entered in, changes. Knowing this, we can save this **value** in a variable. Again, remember this variable will be updated frequently since the **event** fires whenever a key is pressed and released.

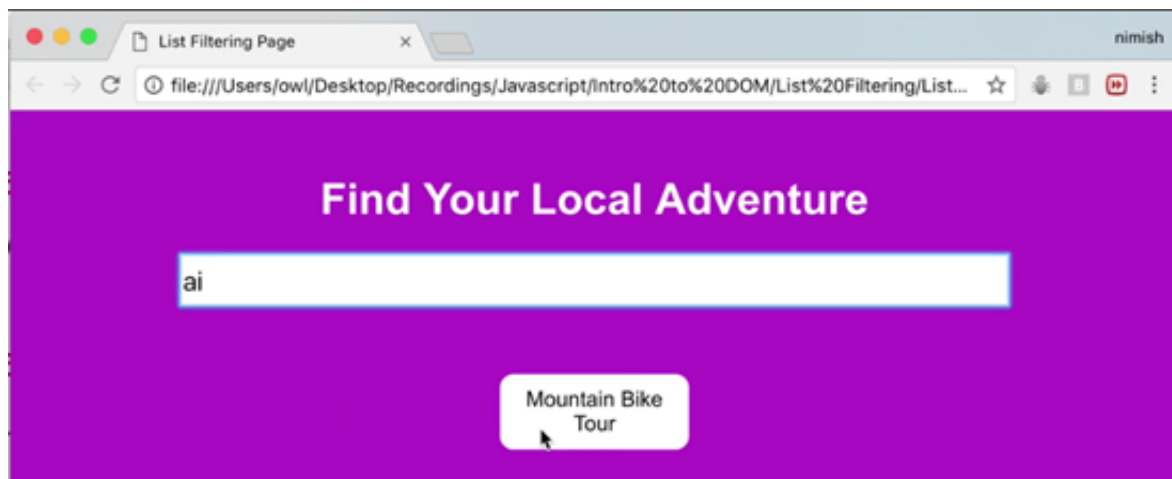
```
var text = searchBar.value;
```

Next, we want to be able to search through our **adventuresList array** of **strings** and check if it matches the **value** we save for the **search bar**. Once again, we need to use a **for loop** so we can check each **string** at a specific **index**. For simplicity, we can even **copy** and **paste** the **loop** from earlier.

Within this **for loop**, we need to use an **if/else statement** to decide whether to set each **item** as **display none** or **display block** depending on how it matches the text. For our **condition** to decide if the **array's string** matches our **text input**, we can use a **function** called **indexOf**. With this function, if the passed in **parameter**, in this case the **text input**, matches the **string**, it will return a **value greater than 0**. If there are no matches, the **value it returns is less than 0**. Thus, we can have our **else** statement catch this former one so that if an item matches, its **display** is set to **block**. Meanwhile, our **if statement** will set any non matching items to **display none**. Since we need a target to set the **style**, we can use the regular **adventures array**, which contains the entire element's HTML, and target the same **index**. This **array's** contents will correlate with the **adventuresList array** so it is able to work fine this way.

```
for (var i = 0; i < adventures.length; i++) {  
  if (adventureList[i].indexOf(text) < 0) {  
    adventures[i].style.display = 'None';  
  } else {  
    adventures[i].style.display = 'Block';  
  }  
}
```

Save the project and test it, as it's now completed! One last thing to note is this search bar is **case sensitive**.





Project Summary

In this project, we created a **list** of items, a search bar, and then styled our page. Next, in our **Javascript**, we created a list of strings from our **list items** using a **for loop**. After, we used **event listeners** and **for loops** in combination to check the value of our search bar and see if it matched any **strings**. Matching strings were shown, while non matching strings were hidden.

With the next and final project of this course, we will work with **client side forms** again.



In the next four lessons, we're going to create our third and final project which consists of an interactive **client side form**. With this form, we'll be simulating a booking system that is able to dynamically show various sorts of **inputs** depending on the answers in other **inputs**. Further, it will be able to react to certain **combinations** of **inputs**. Like the previous project, we'll be working a lot with **event listeners** on **inputs** to create this functionality. Keep in mind this form is **client side only**, as the **server side PHP** aspects are beyond the scope of this course.

Before we move on, be sure to create a **new project folder** called **Interactive Form Code Files**. Additionally, create a **new HTML file** called **Interactive Forms Page** in **ATOM** (or other code editor) and save it into this project folder. You may also find it helpful to open up your browser to test the file as we go.

In the next lesson, we'll begin writing the **HTML skeleton** for our project.

In this lesson, we're going to build the **HTML** backbone of our "Interactive Forms Page". As mentioned, this project will simulate a booking form. We will take information from the user, and then simulate whether the form was submitted or whether something was wrong.

Basic HTML

As usual, we will start off with **head** and **body tags**. In the **body tags**, we will set up a **div container** to contain our entire form. In this **container**, we'll begin our form which we will give a unique **ID**.

```
<head></head>
<body>
  <div class='container'>
    <form id='saveForm'>

      </form>
    </div>
  </body>
```

The first three **inputs** that we'll add to our form are **text inputs** to get the user's name, address, and age. These won't be anything too special since there are ways we can access these elements without additional **classes** or **IDs**. However, our fourth **input** will be a **checkbox** that wants to know if the user has pets. Thus, instead of **text**, we set the **type** to **checkbox**. We also want to add an **ID** to this input field since we'll be looking for it later outside the form.

```
Name: <br><input type='text'><br>
Address: <br><input type='text'><br>
Age: <br><input type='text'><br>
Pets: <br><input id='petCheckbox' type='checkbox'><br>
```

(**Note:** So that our form has a more top to bottom look, we'll be use lots of break tags to separate elements into separate lines).

For part of our page's functionality, we want a **dropdown** menu to appear that contains types of pets whenever the user has the **checkbox checked**. However, if **unchecked**, we want this **dropdown** menu hidden. Since we want to hide the entire menu, we're going to enclose the entire thing in its own **div** with an **ID** of **petSelection**. Using the **select** and **option tags**, we can create our **dropdown** menu within it.

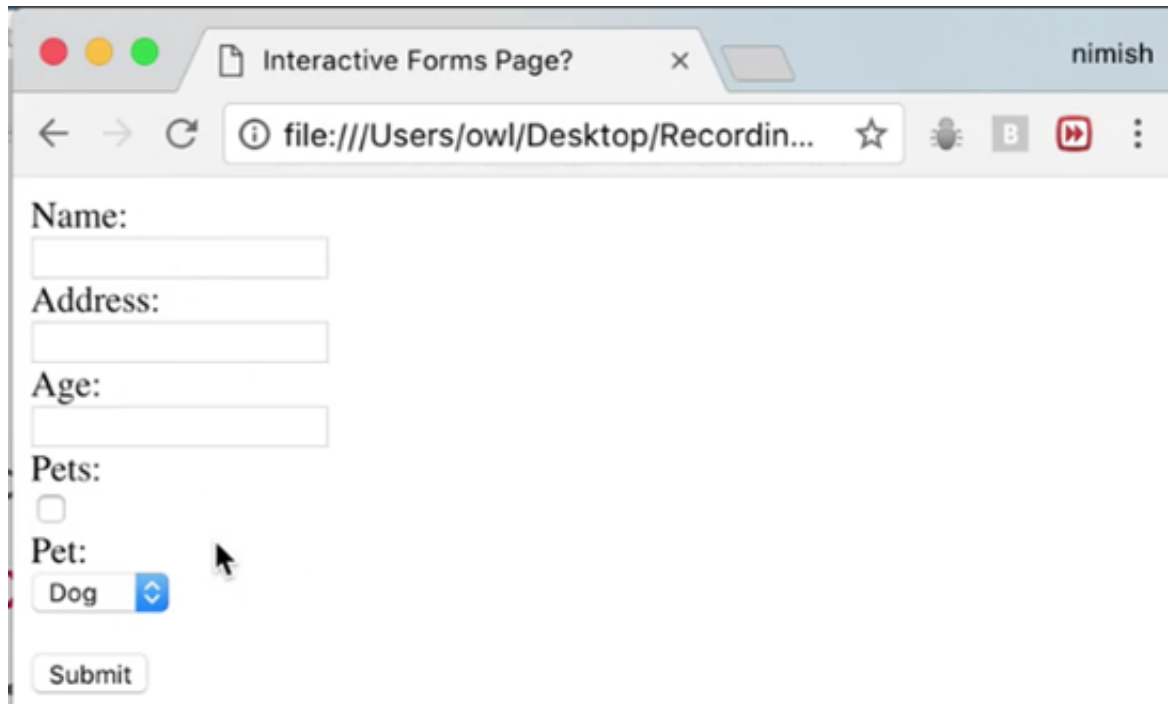
```
<div id='petSelection'>
  Pet: <br><select>
    <option value=''>Dog</option>
    <option value=''>Cat</option>
    <option value=''>Bird</option>
    <option value=''>Other</option>
  </select><br>
</div>
```

Last, before our **closing form tag**, we want to add a **submit type input** that allows us to submit

the form. Since we also want particular access to this, we'll also give it an **ID**.

```
<br><input id='saveButton' type='submit' value='Save'>
```

Our **HTML** is all done.



The screenshot shows a web browser window with the title 'Interactive Forms Page?'. The address bar shows a file path: 'file:///Users/owl/Desktop/Recordin...'. The form contains the following elements:

- Name:
- Address:
- Age:
- Pets: ☐
- Pet:
- Submit:

In the next lesson, we'll add our **CSS styling** to the page.

In this lesson, we're going to **style** our form with **CSS**.

Quick HTML Fix

Before we begin, last lesson we said our **text inputs** didn't need anything special. While this is true for our **Javascript**, we do want them to have classes for our **CSS styling**. As such, we're going to add **classes** to them as seen below:

```
Name: <br><input class='textInput' type='text'><br>
Address: <br><input class='textInput' type='text'><br>
Age: <br><input class='textInput' type='text'><br>
```

CSS Styling

Now we can style our page. In the **head tags**, open up some **style tags** so we can start. In these new **style tags**, we'll start off quickly mostly by changing the **body's padding** and **font** to **Arial**.

```
body {
  padding: 40px;
  font-family: Arial;
}
```

Next, we're going to style our **div container** where most of our **CSS** will be. We want to consistently **center** our **container** in the middle of the screen. This time, we will use a longer method that involves **absolute positioning** and setting all the **directions** to 0. **Margin** also needs to be set to **auto** for this to work. Then, besides **padding**, we'll give the **container** a fixed **width** and **height**, add a **dark purple border**, and change the **background** to be **teal**.

```
.container {
  position: absolute;
  left: 0;
  right: 0;
  top: 0;
  bottom: 0;
  margin: auto;
  padding: 10px;
  width: 200px;
  height: 300px;
  border: thick solid #9900ff;
  background-color: #00ffaa;
}
```

A form with a purple border. It contains the following elements: a 'Name:' label followed by a text input; an 'Address:' label followed by a text input; an 'Age:' label followed by a text input; a 'Pets:' label followed by a checkbox; a 'Pet:' label followed by a dropdown menu showing 'Dog'; and a 'Save' button at the bottom.

We will work on elements in our form next. For our **text inputs**, we're going to space them out better and set the **width** to span the whole **container**. Meanwhile, we'll also take this time to add a **margin** to our **dropdown** and to set the **display** as **none** so it's properly hidden when the page loads.

```
.textInput {  
  margin-bottom: 10px;  
  width: 100%;  
}  
#petSelection {  
  margin-top: 10px;  
  display: none;  
}
```

Last, we want to style the **submit button**. Using **absolute position**, we're going to fix the button to the bottom right. We'll also change the **background** to **purple** and change the text to **white**.

```
#saveButton {  
  position: absolute;  
  right: 20px;  
  bottom: 20px;  
  background-color: #9900ff;  
  color: white;  
}
```

Our form is now looking better!



Name:

Address:

Age:

Pets:
☐

Save

In the next lesson, we'll begin writing the **Javascript** for our form.

In this lesson, we're going to create the **Javascript** functionality that **shows** and **hides** the **dropdown** menu based on whether the **checkbox** is **checked** or not.

Checkbox Functionality

At the bottom of the **body**, make sure to first add your **script tags** to contain the **Javascript**. The first things we need to do are get the **checkbox** from the **DOM** and get the **dropdown** from the **DOM**. Since we gave the **checkbox** and the **div** containing the **dropdown** unique **IDs**, we can use those to select and save them.

```
var checkbox = document.getElementById('petCheckbox');  
var selectionDiv = document.getElementById('petSelection');
```

Now that we have **pointers** to the correct elements, we can add an **event listener** to our **checkbox**. The type of **event** we'll be listening for is **change**. With this **event type**, the **listener** is triggering whenever the **value** of the item has changed. For a **checkbox**, whenever the **checkbox** is **checked**, the **value** for this is changed to **true** in the background. If you check it and uncheck it, the **value** is one again **changed**, with **unchecked** being false.

```
checkbox.addEventListener('change', function(event){  
  
});
```

Knowing this, in our **event listener**, all we need to do is use an **if/else statement** to determine whether that **checked value** is true or not. **If the checked value is true**, we will have our **div** with the **dropdown** change its **display** to **block**. **Else**, if **checked isn't true**, our **dropdown div** will **display** to **none**.

```
checkbox.addEventListener('change', function(event){  
    if (checkbox.checked == true) {  
        selectionDiv.style.display = 'Block';  
    } else {  
        selectionDiv.style.display = 'None';  
    }  
});
```

You can now **save** the page and test it to see that it's working.



Name:

Address:

Age:

Pets:
☒

Pet:

In the next and final lesson, we will implement the **JavaScript** that will simulate our form submission.

In this final lesson of the course, we're going to set up the functionality of our form **submission**. Through this, we want to take in the information in all the fields and print various **alerts** that specify whether the submission was successful or not. Remember that this is a **client side form** so no actual data will be sent.

Quick HTML Fix

Once again, we need to add something to our **HTML** before we jump in to the lesson's contents. In the **dropdown** menu, we need to give our **options** actual **values** so that we can retrieve the **value attribute** in our **Javascript**. Your code should look like the following with the change:

```
<option value='Dog'>Dog</option>
<option value='Cat'>Cat</option>
<option value='Bird'>Bird</option>
<option value='Other'>Other</option>
```

Event Listener and Variables

In the **script tags** underneath our **checkbox** functionality, we're going to add in our **submit** button functionality. The first task we need to perform is to get our **form** by its **ID** (saveForm) and add an **event listener** to it. The **event** we'll be listening for is **submit** since that is the **type** we specified for our **input**. If this was a **server side form**, this **submit** would probably make a call to the **PHP**. For our purposes, we're just going to add our own simulated functionality.

```
var saveForm = document.getElementById('saveForm');
saveForm.addEventListener('submit', function(event){

});
```

The rest of our code will go within the **event listener**. Like in our previous lesson on forms, we first need to **prevent** the **default** behavior so that the page doesn't reload and clear the fields.

```
event.preventDefault();
```

Now we want to get the **value** for each of our input fields and save them. Despite not giving each **input** a unique **ID**, by accessing our form (via the **saveForm variable**) and using the **elements** function, we can access each **input** by an **index**. The **name input** will be **index 0**, the **address input** will be **index 1**, and so forth. For our **dropdown**, we'll just use a **variable** (petText) to hold a default value.

```
var nameText = saveForm.elements[0].value;
var addressText = saveForm.elements[1].value;
var ageText = saveForm.elements[2].value;
var doesHavePets = saveForm.elements[3].checked;
var petText = 'No pets';
```

Notice in the code above that our **text inputs** use **.value** to get the **value**, while our

checkbox uses **.checked** to check the **true or false value** of the attribute.

Setting up the Alerts

With all our **values** able to be checked and saved, we can now pass those **values** through a series of **if statements** to determine which **alert** to send. Our first few checks will be to see if the **text inputs** are filled out. For each **saved text input**, we will check to see if the **value** it saved is an **empty string**. If it is, we'll print an **alert** saying the user must fill that field out. Then, using **return**, we can break out of the **function** so that the user can't **submit** the form without them.

```
if (nameText == '') {  
    alert('Please enter a name');  
    return;  
}  
if (addressText == '') {  
    alert('Please enter an address');  
    return;  
}  
if (ageText == '') {  
    alert('Please enter an age');  
    return;  
}
```

The next check we want to do is to see **if** the user has **checked** the **checkbox**. If the user has **checked** it, we want to change the **petText variable** to be the **value** that they selected from the **dropdown**. Again, we can use the **index** of the **input** in the form to access the **value** in the **dropdown**.

```
if (doesHavePets) {  
    petText = saveForm.elements[4].value;  
}
```

For the sake of demonstration, we're going to pretend our dog bookings are full so we can't accept anyone with dogs. Since we just made a way to save the **petText**, we can now check the **string** to see if it says "Dog". If it does, we'll **alert** the user we can't accept them and **return** out of the **function**.

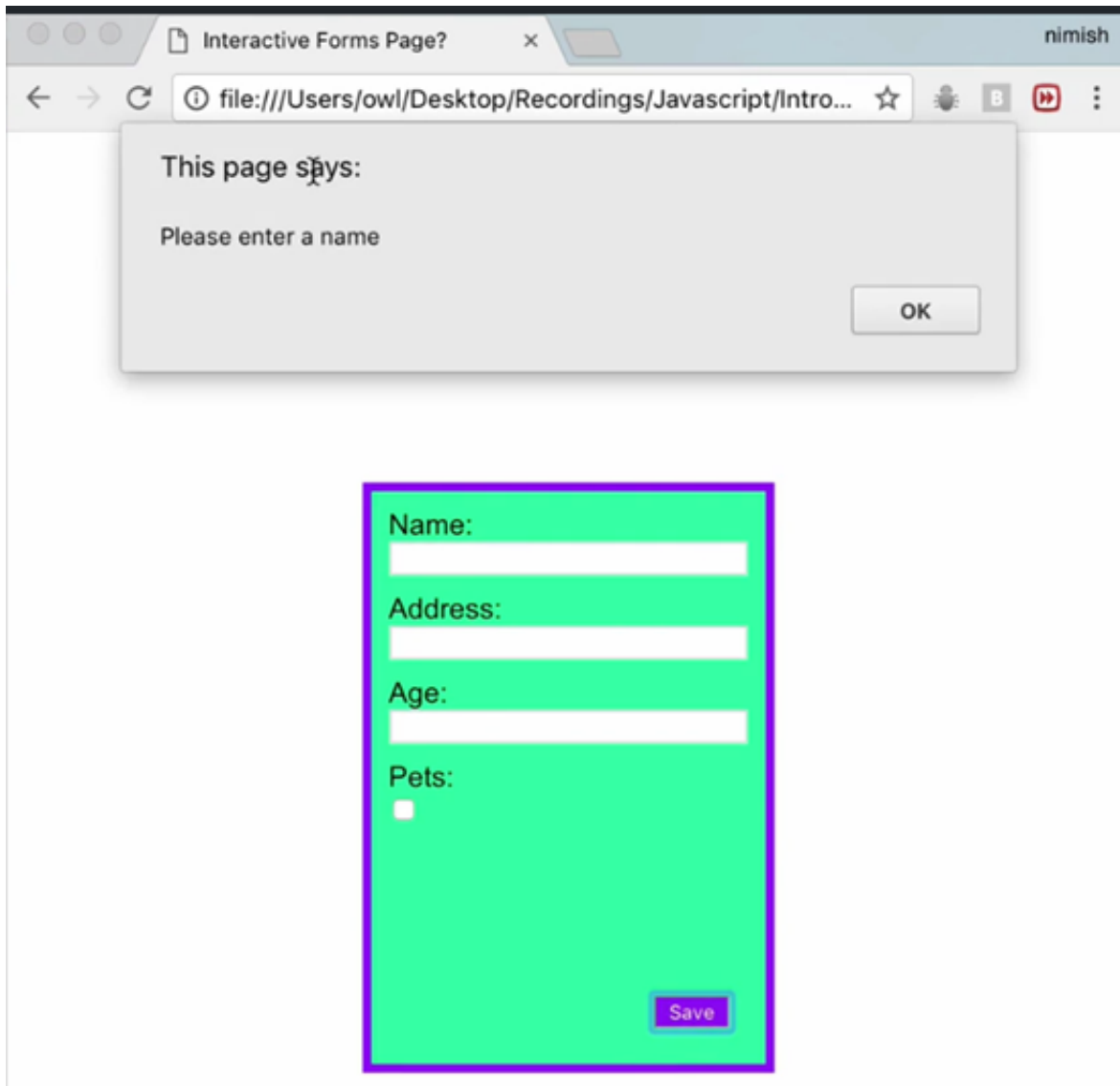
```
if (petText == 'Dog') {  
    alert('Sorry, our dog capacity is full :(');  
    return;  
}
```

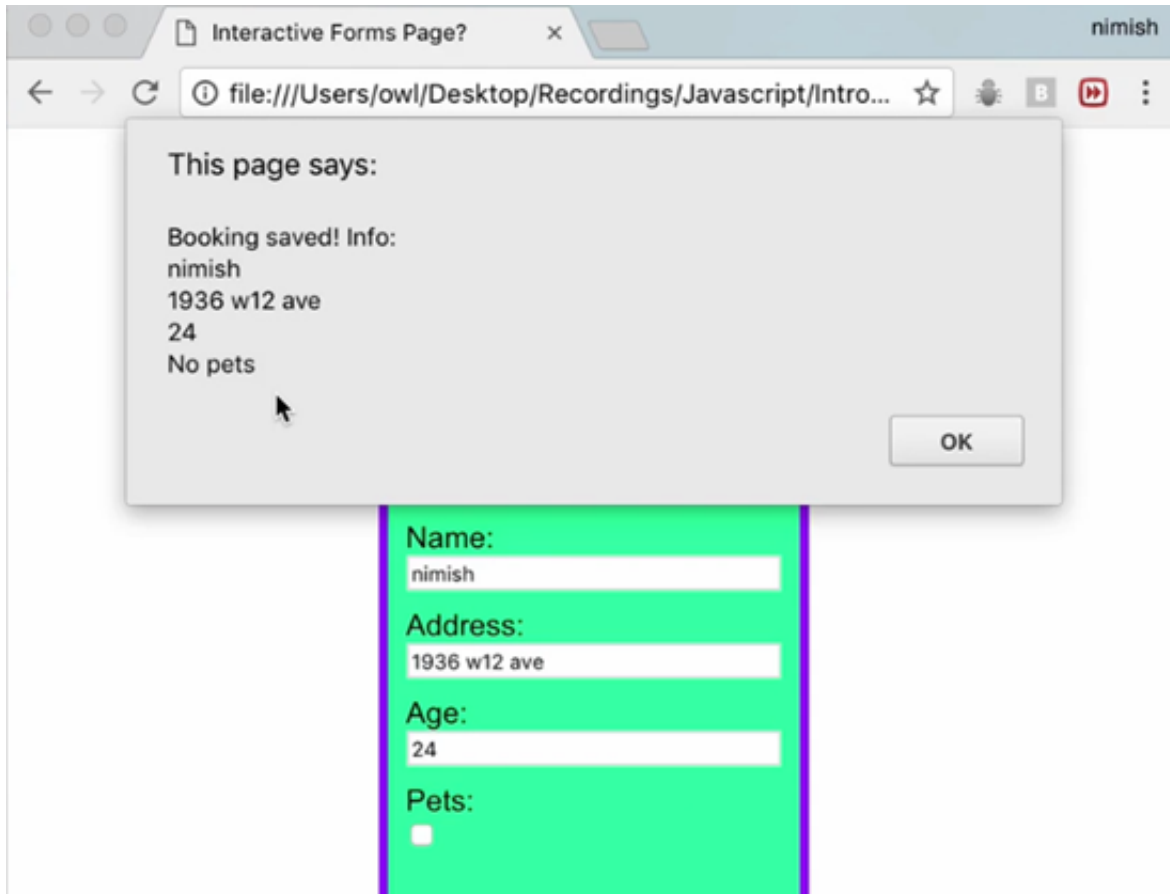
The last thing we want to add is an **alert** for if the user passed through all the checks and successfully submitted the form. For this **alert**, we'll create a **concatenated string** that prints back the **values** they entered. Using **\n**, we can put everything on a new line so it looks nice.

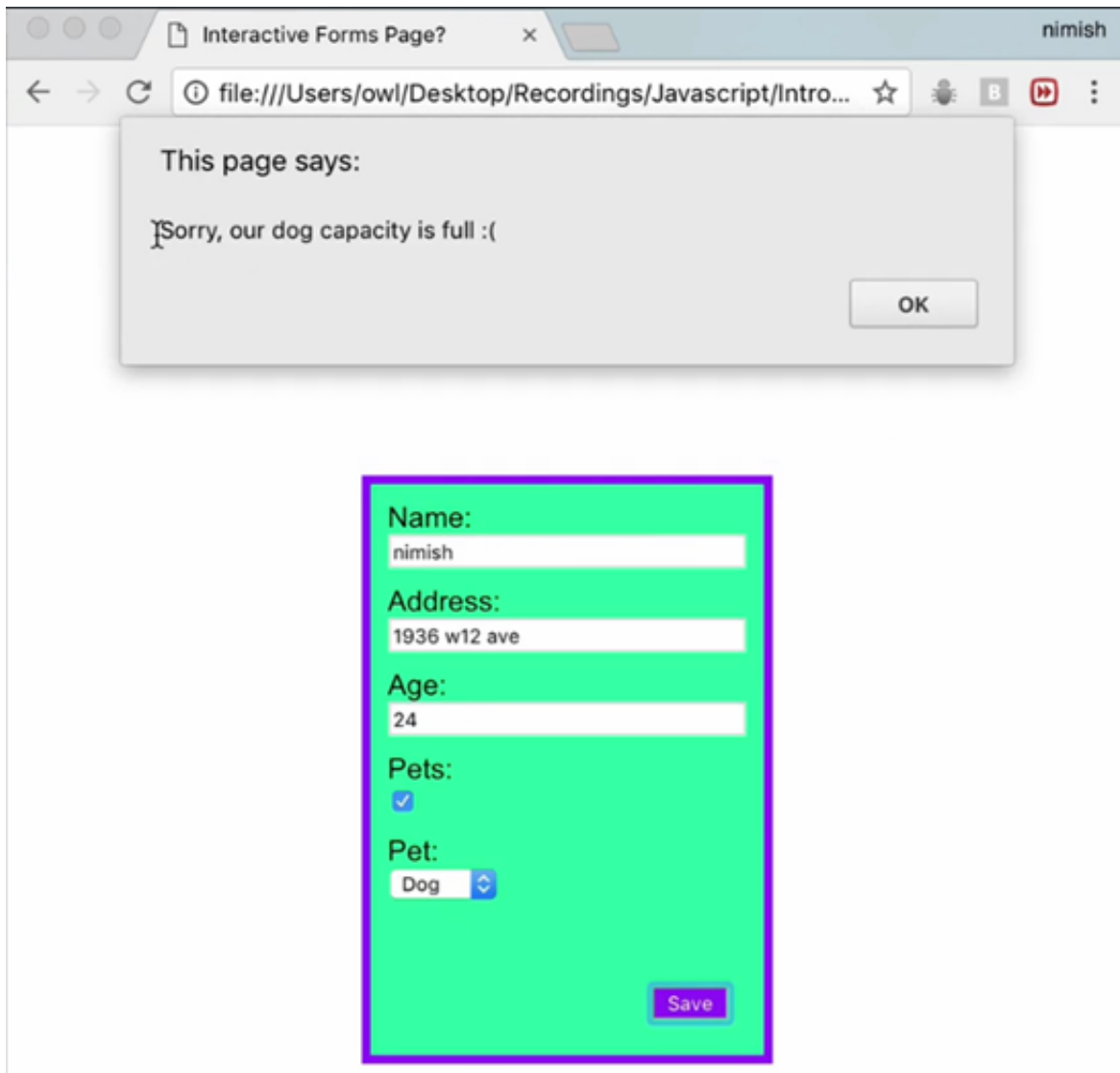
```
alert('Booking saved! Info:\n' +  
    nameText + '\n' +  
    addressText + '\n' +
```

```
ageText + '\n' +  
petText  
);
```

Save the file and test it out! You've now completed the project.







Project Summary

Through this final project, we created a simple form with **text inputs**, a **checkbox**, and dynamic **dropdown**. In our **Javascript**, we enabled our **checkbox's checked value** to control whether to show the **dropdown**. Using an **event listener**, we also simulated various types of **form validation** using **alerts**.

Congratulations! You've now completed **Zenva's Create Interactive Pages with Javascript and the DOM API** course! Through this course, we've learned extensively how to access elements in the **DOM** and manipulate them using **Javascript** to create more interactivity. The methods we learned throughout the course can be used in a variety of robust ways to create dynamic websites, so experiment with the skills you've learned here.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Flashcards Page

Found in Project/02. Spanish Flashcards/Spanish Flashcards Code Files

The code creates a responsive flashcard interface for language learning. Each card displays an image and an English word, and reveals its Spanish equivalent when the “Show” button is clicked. The revealed word is hidden again when the button, which now reads “Hide”, is clicked again. This is achieved using CSS for styling and JavaScript for functionality.

```
<head>
  <style>
    body {
      padding: 40px;
      background: #5f9fff;
      font-family: courier;
    }
    .container {
      display: flex;
      flex-flow: row wrap;
      justify-content: space-around;
    }
    .card {
      flex: none;
      margin-top: 10px;
      padding: 20px;
      border-radius: 10px;
      background: white;
    }
    .card-title, .card-answer {
      margin-top: 10px;
      margin-bottom: 10px;
      text-align: center;
      font-weight: bold
    }
    .card-button {
      width: 100%;
      margin-top: 10px;
      border: 1px solid #0070bb;
      background: white;
      color: #0070bb;
    }
    .card-answer {
      display: none;
    }
  </style>
</head>
<body>
```

```
<div class='container'>
  <div class='card'>
    <div></div>
    <div class='card-title'>Apple</div>
    <button class='card-button'>Show</button>
    <div class='card-answer'>Manzana</div>
  </div>
  <div class='card'>
    <div></div>
    <div class='card-title'>Dog</div>
    <button class='card-button'>Show</button>
    <div class='card-answer'>Perro</div>
  </div>
  <div class='card'>
    <div></div>
    <div class='card-title'>City</div>
    <button class='card-button'>Show</button>
    <div class='card-answer'>Ciudad</div>
  </div>
  <div class='card'>
    <div></div>
    <div class='card-title'>Chair</div>
    <button class='card-button'>Show</button>
    <div class='card-answer'>Silla</div>
  </div>
</div>

<script>
  /*var firstButton = document.getElementsByClassName('card-button')[0];
  var firstAnswer = document.getElementsByClassName('card-answer')[0];
  firstButton.addEventListener('click', function(event) {
    if (firstButton.innerHTML == 'Show') {
      firstAnswer.style.display = 'Block';
      firstButton.innerHTML = 'Hide';
    } else if (firstButton.innerHTML == 'Hide') {
      firstAnswer.style.display = 'None';
      firstButton.innerHTML = 'Show';
    }
  })*//

  var cards = document.getElementsByClassName('card');
  for (var cardIndex in cards) {
    var button = cards[cardIndex].querySelector('.card-button');
    button.addEventListener('click', function(event){
      var currentButton = event.target;
      var currentCard = currentButton.parentNode;
      var currentAnswer = currentCard.querySelector('.card-answer');
      if (currentButton.innerHTML == 'Show') {
        currentAnswer.style.display = 'Block';
        currentButton.innerHTML = 'Hide';
      } else if (currentButton.innerHTML == 'Hide') {
        currentAnswer.style.display = 'None';
        currentButton.innerHTML = 'Show';
      }
    })
  }
}
```

```
    }  
  
    </script>  
</body>
```

List Filtering Page

Found in Project/03. List Filtering/List Filtering Code Files

The code creates a web page for searching local adventures. It displays a list of available options, and as the user types into the search bar, the list is filtered to only display adventures that include the search text. This is accomplished using CSS for styling and JavaScript for search functionality.

```
<head>  
  <style>  
    body {  
      padding: 40px;  
      background: #b10dc9;  
      font-family: Arial;  
    }  
    h1 {  
      text-align: center;  
      color: white;  
    }  
    .search-container {  
      display: flex;  
      justify-content: space-around;  
    }  
    #search-bar {  
      width: 80%;  
      height: 40px;  
      font-size: 20px;  
    }  
    .adventures-container {  
      display: flex;  
      flex-flow: row wrap;  
      justify-content: space-around;  
      margin: 40px;  
    }  
    .adventure {  
      width: 120px;  
      padding: 10px;  
      margin-top: 10px;  
      border-radius: 10px;  
      text-align: center;  
      background-color: white;  
    }  
  </style>  
</head>  
<body>  
  <h1>Find Your Local Adventure</h1>  
  <div class='search-container'><input id='search-bar' type='text'></div>  
  <div class='adventures-container'>
```



```
<div class='adventure'>Mountain Bike Tour</div>
<div class='adventure'>Hinterlands Tour</div>
<div class='adventure'>Sky Diving</div>
<div class='adventure'>Local Food Festival</div>
<div class='adventure'>Sunday Markets Tour</div>
<div class='adventure'>Food Tasting Tour</div>
<div class='adventure'>Snorkelling Tour</div>
<div class='adventure'>Forest Experience</div>
</div>

<script>
  var adventureList = [];
  var adventures = document.getElementsByClassName('adventure');
  for (var i = 0; i < adventures.length; i++) {
    var adventureName = adventures[i].innerHTML;
    adventureList.push(adventureName);
  }

  var searchBar = document.getElementById('search-bar');
  searchBar.addEventListener('keyup', function(event){
    var text = searchBar.value;
    for (var i = 0; i < adventures.length; i++) {
      if (adventureList[i].indexOf(text) < 0) {
        adventures[i].style.display = 'None';
      } else {
        adventures[i].style.display = 'Block';
      }
    }
  });
</script>
</body>
```

Interactive Forms Page

Found in Project/04. Interactive Forms/Interactive Forms Code Files

This code creates a form asking for a person's name, address, age, and pet information and validates the user input. It contains an interactive pet section that only appears if the 'Pets' checkbox is selected and provides different alerts based on the user's input. After validation checks, it shows an alert with the saved booking information.

```
<head>
  <style>
    body {
      padding: 40px;
      font-family: Arial;
    }
    .container {
      position: absolute;
      left: 0;
      right: 0;
      top: 0;
      bottom: 0;
```



```
    margin: auto;
    padding: 10px;
    width: 200px;
    height: 300px;
    border: thick solid #9900ff;
    background-color: #00ffaa;
}
.textInput {
    margin-bottom: 10px;
    width: 100%;
}
#petSelection {
    margin-top: 10px;
    display: none;
}
#saveButton {
    position: absolute;
    right: 20px;
    bottom: 20px;
    background-color: #9900ff;
    color: white;
}
</style>
</head>
<body>
    <div class='container'>
        <form id='saveForm'>
            Name: <br><input class='textInput' type='text'><br>
            Address: <br><input class='textInput' type='text'><br>
            Age: <br><input class='textInput' type='text'><br>
            Pets: <br><input id='petCheckbox' type='checkbox'><br>
            <div id='petSelection'>
                Pet: <br><select>
                    <option value='Dog'>Dog</option>
                    <option value='Cat'>Cat</option>
                    <option value='Bird'>Bird</option>
                    <option value='Other'>Other</option>
                </select><br>
            </div>
            <br><input id='saveButton' type='submit' value='Save'>
        </form>
    </div>

    <script>
        var checkbox = document.getElementById('petCheckbox');
        var selectionDiv = document.getElementById('petSelection');
        checkbox.addEventListener('change', function(event){
            if (checkbox.checked == true) {
                selectionDiv.style.display = 'Block';
            } else {
                selectionDiv.style.display = 'None';
            }
        });

        var saveForm = document.getElementById('saveForm');
```



```
saveForm.addEventListener('submit', function(event){
    event.preventDefault();
    var nameText = saveForm.elements[0].value;
    var addressText = saveForm.elements[1].value;
    var ageText = saveForm.elements[2].value;
    var doesHavePets = saveForm.elements[3].checked;
    var petText = 'No pets';
    if (nameText == '') {
        alert('Please enter a name');
        return;
    }
    if (addressText == '') {
        alert('Please enter an address');
        return;
    }
    if (ageText == '') {
        alert('Please enter an age');
        return;
    }
    if (doesHavePets) {
        petText = saveForm.elements[4].value;
    }
    if (petText == 'Dog') {
        alert('Sorry, our dog capacity is full :(');
        return;
    }
    alert('Booking saved! Info:\n' +
        nameText + '\n' +
        addressText + '\n' +
        ageText + '\n' +
        petText
    );
});
</script>
</body>
```